

TEAM 3 · FINAL REPORT

AdNova 통합 최종 보고서

상품 사진 한 장을 광고 자산으로 바꾸는 AI 서비스

상세보고서

유연정 · 한의정 · 김범수 · 정봄

프로젝트 기간 2026.06.30 — 2026.07.30

목차 및 문서 구성

1. 프로젝트 개요와 문제 정의
2. 요구사항·협업 운영·전체 아키텍처
3. 기획·PM·풀스택·인증·브랜드·배포 — 유연정
4. AI 생성 파이프라인·검증·비용·운영 — 한의정
5. 백엔드·DB·관리자·DevOps — 김범수
6. 사용자·관리자 프론트엔드와 QA — 정봄
7. 통합 보안·데이터 무결성·검증 결과
8. 종합 한계와 다음 단계

부록. 개인보고서 증빙 이미지

1. 프로젝트 개요와 문제 정의

AdNova는 소상공인이 상품 사진을 업로드하고 원하는 분위기와 광고 용도를 선택하면 제품 정체성을 유지한 광고 이미지와 플랫폼별 문구를 생성하는 웹 서비스다. 단일 L4 GPU와 제한된 API 예산에서 로컬 모델과 상용 API를 목적별로 분리하고, 인증·저장·관리자·결제 데모·공유·QA·배포까지 서비스 전체를 구현했다.

- 입력: 상품 사진, 상품명, 추가 요청, 스타일·용도
- 출력: 광고 이미지와 Instagram·Facebook·X·Threads 별 문구
- 주요 용도: SNS, 배너, 카드뉴스, 상세페이지
- 서비스 기능: 인증, 생성, 저장·히스토리, 다운로드·공유, 구독·크레딧, 관리자 운영

1.1 핵심 설계 원칙

광고 생성은 미적 품질만으로 평가할 수 없다. 제품의 라벨, 형상, 재료, 개수와 조리 상태가 입력과 다르면 광고로 사용할 수 없기 때문이다. 따라서 프로젝트는 '제품 정체성 보존'과 '광고 연출 품질'의 균형을 핵심 기준으로 삼았다.

1.2 팀 역할

구성원	담당	핵심 결과
유연정	기획·PM·풀스택·인증·브랜드·배포	서비스 방향·협업 체계, 인증 API·OAuth·OTP, Next.js, 도메인·HTTPS
한의정	AI·프롬프트·생성 파이프라인	무드·자유입력·템플릿 3 경로, 로컬/API 비교, 품질 개선·카피 Judge
김범수	백엔드·DB·DevOps	이미지·히스토리·크레딧·관리자 API, 보안, Docker·데이터 이전
정봄	프론트·관리자·QA	사용자·관리자 화면, 실제 API 연결, 디자인 통일, QA 127 건

2. 요구사항·협업 운영·전체 아키텍처

초기에 BR·FR·NFR, API 명세, User/System/AI Flow, Architecture, 일정과 Git 규칙을 만들었다. 공통 파일
누락 경험을 계기로 develop 선병합, 작업 전 pull, 기능 단위 PR, 병합 공지, 24 시간 이내 코드 리뷰
원칙을 확정했다.

기간	핵심 변화
6/30~7/3	상품 사진 기반 MVP 확정, Auth·DB·AI·UI 골격
7/6~7/10	파트 연동, 로컬 모델 실험, HTML UI 전환
7/13~7/17	CI·브랜드·OAuth·도메인·관리자·LangChain Judge
7/20~7/24	Next.js 통합, 템플릿·SNS·TOTP·이메일 OTP·QA
7/27~7/28	최종 QA, Docker·AI 배포 정리, 보고서·발표 준비

2.1 시스템 계층과 책임

계층	책임	기술
Frontend	인증·업로드·생성·히스토리·관리자 UI	Next.js App Router
Web API	검증·권한·DB·라우팅·저장·결제 데모	FastAPI, SQLAlchemy, Pydantic
AI Worker	로컬 이미지 편집과 용도별 생성	FLUX.1 Kontext, Pillow, BiRefNet
External AI	분석·카피·자유입력·템플릿	GPT-5.4-mini, GPT-image-2
Infra	TLS·프록시·컨테이너·GPU 서비스	Nginx, Docker Compose, systemd, GCP

외부 요청은 Nginx 를 거쳐 Next.js 프론트와 FastAPI 웹 백엔드로 전달된다. 일반 웹 스택은 Docker
Compose 로 고정하고, CUDA 와 대형 모델이 필요한 GPU 생성 워커는 systemd 로 분리했다. SQLite 와
결과 파일은 영구 runtime 경로에 보존했다.

3. 기획·PM·풀스택·인증·브랜드·배포 — 유연정

원문 개인보고서의 본문·표를 통합 문서 체계에 맞춰 수록하였다. 담당: 유연정

요약

아이디어를 실제 사용 가능한 서비스 구조로 연결한 PM·풀스택 수행 기록

AdNova 는 소상공인이 제품 사진 한 장과 원하는 분위기를 입력하면 광고 이미지와 플랫폼별 문구를 생성하는 서비스다. 본 프로젝트에서 나는 PM 으로서 서비스 방향, 요구사항, 협업 체계를 설계했고, 인증 백엔드부터 사용자 화면과 배포 환경까지 풀스택으로 직접 구현했다.

역할	주요 수행	결과
기획·PM	요구사항·API·Flow·일정·역할·회의 운영	상품 이미지 기반 MVP 와 협업 기준 확정
인증·보안	JWT/bcrypt, Google·Kakao·Naver OAuth, 이메일 OTP	다중 인증 경로와 가입 검증 흐름 완성
풀스택·브랜드	인증 API, Streamlit→HTML→Next.js, 디자인 시스템·반응형	백엔드 인증부터 사용자 화면까지 통합
배포·운영	도메인·HTTPS, GCP 24/7, CI·CODEOWNERS, QA 대응	외부 접속 가능한 서비스와 운영 기반 확보

1. 프로젝트 출발점과 문제 정의

6 월 30 일 초기 기획은 단순한 텍스트 기반 이미지 생성이었지만, 실제 소상공인에게 필요한 것은 자신의 상품을 유지한 채 광고처럼 보이게 만드는 기능이라고 판단했다. 이에 핵심 입력을 '상품 사진'으로 바꾸고, 제품 정체성 보존을 서비스 원칙으로 제안했다.

- 핵심 고객: 촬영·디자인 비용과 시간이 부담스러운 소상공인
- 핵심 가치: 상품 사진 한 장으로 광고 이미지와 SNS 문구 생성
- MVP 우선순위: 상품 이미지 기반 광고 생성 → 저장·이력 → 공유
- 제외·후순위: 스케치 생성, 텍스트 전용 생성, 완전 자동 게시

2. PM 과 협업 구조 설계

프로젝트 초기에 Notion 협업 공간, 일정 데이터베이스, 요구사항·기능·비기능 명세, API 명세, 사용자·시스템·AI 처리 Flow Chart, 아키텍처, Git 규칙과 README 초안을 만들었다. 이후 데일리 스크럼을 통해 역할과 차단 이슈를 매일 확인했다.

협업 장치	적용 내용
Git 전략	feature → develop → main, 기능 단위 커밋, PR 기반 병합
충돌 예방	작업 전 pull, develop 병합 전 팀 공지, 공통 파일 책임 명확화
품질 관리	24 시간 내 코드 리뷰, 이슈 등록, QA 재현 조건 기록
역할 구분	유연정 기획·PM·풀스택·인증·브랜드·배포, 김범수 BE·DevOps, 한의정 AI, 정봄 FE·Admin·QA

3. 인증과 사용자 진입 흐름

초기 인증 기반은 bcrypt 비밀번호 해시와 JWT Access Token 으로 구성했다. 회원가입 스키마에는 이메일·아이디·비밀번호 규칙을 두고, DB 담당자와 범위를 분리해 공통 모델 충돌을 줄였다.

- Google·Kakao·Naver OAuth 회원가입·로그인 연결 및 리다이렉트 오류 수정
- 이메일·아이디 실시간 중복 확인으로 가입 실패를 사전 방지
- AWS SES 기반 DKIM·SMTP 인프라와 6 자리 이메일 OTP 구현
- OTP 5 분 만료, 60 초 재발송 대기, 최대 5 회 검증 제한
- 온보딩 화면, 프로필 아바타, 이용약관 모달과 비밀번호 변경 흐름

외부 서비스 승인이나 테스터 제한이 있는 기능은 정상 경로와 시연 경로를 구분했다. 코드 삭제 대신 환경 변수 기반으로 인증 요구 여부를 바꿀 수 있게 하여 운영 전환 가능성을 남겼다.

4. 프론트엔드와 브랜드 고도화

초기 Streamlit 화면은 빠른 기능 확인에는 유리했지만 무겁고 화면 제어가 제한적이었다. UI 피드백을 반영하면서 HTML 로 전환했고, 최종적으로 Next.js App Router 구조로 로그인·회원가입·Dashboard·Studio·내 광고·고객센터를 통합했다.

단계	변화	판단
Streamlit	기능 빠대와 API 연동 확인	빠른 MVP 검증

HTML/JS	반응 속도와 디자인 자유도 개선	중간 발표용 사용자 흐름 정리
Next.js	라우팅·상태·반응형·컴포넌트 통합	최종 서비스 구조로 채택

- 골드·다크 브랜드 테마, 로고·파비콘·소셜 프리뷰 제작 및 적용
- 이미지 드래그 앤 드롭, 5MB 업로드와 실제 파일 검증 안내
- Instagram·Facebook·X·Threads 별 AI 문구 동기화와 복사·작성 화면 이동
- 모바일·태블릿 반응형, 다크 테마, 아바타 동적 그라데이션
- 사용자 화면과 관리자 화면의 디자인 언어 정렬

5. CI/CD, 서버와 운영 기반

GCP 에서 프론트엔드와 백엔드를 상시 실행하고 adnova.iridescentseraphim.org 도메인과 HTTPS 를 연결했다. CI 에는 Ruff Lint/Format 과 Import Smoke Test 를 넣고, CODEOWNERS 로 경로별 리뷰 책임을 정했다.

- GCP 방화벽과 서비스 포트 점검, 외부 접속 확인
- 모델 중복 다운로드로 인한 디스크·OOM 원인 분석 및 공유 폴더 전략 제안
- 브랜치 보호, PR 대상 브랜치, 자동 배포 충돌 등 운영 이슈 조정
- 서버 디스크 100% 근접 상황을 진단하고 팀 정리 필요성을 공지
- QA 에서 발견된 인증·SNS·대시보드 문제를 직접 수정하거나 담당자에게 재현 정보 전달

6. 주요 문제 해결

문제	원인	해결
Feature 브랜치 파일 누락	공통 파일이 develop 에 선병합되지 않음	공통 변경 선병합 후 feature 생성 규칙
OAuth 중복·리다이렉트 오류	플랫폼별 콜백과 상태 처리 차이	플랫폼별 경로 분리와 로딩·취소 상태 정리
HTTP 413 업로드 실패	서버·프론트 제한 불일치	5MB 기준 통일과 검증 로직 추가
GPU OOM·디스크 부족	모델 중복과 VRAM 경합	공유 모델 경로와 실행 역할 분리 제안
SNS 자동 게시 제약	권한·계정·심사 조건 상이	1 차 범위를 문구 생성·복사·작성 화면 이동으로 제한

7. 성과와 배운 점

- 기획 문서를 구현 기준으로 연결하여 파트 간 의존성을 조기에 드러냈다.
- 인증 백엔드·프론트엔드·브랜드·배포를 풀스택으로 직접 다루며 PM 의사결정을 코드와 운영 결과로 검증했다.
- 기술적으로 가능한 범위와 실제 플랫폼 정책상 가능한 범위를 구분했다.
- 서비스 완성도는 기능 수보다 사용자 흐름, 데이터 보존, 오류 안내, 배포 안정성의 합이라는 점을 확인했다.

8. 한계와 다음 단계

- OAuth 앱 심사와 실제 SNS 게시 권한 검증
- SMTP 운영 승인 후 이메일 인증·비밀번호 재설정 E2E 테스트
- 모니터링·백업·비밀키 회전 및 정식 배포 파이프라인 강화
- QA 자동화와 접근성·모바일 회귀 테스트 확대
- PM 관점의 지표 정의: 생성 성공률, 재생성률, 공유 전환, 사용자당 비용

이번 프로젝트에서 가장 큰 기여는 팀의 각 기능을 하나의 서비스 경험으로 연결한 것이다. 앞으로는 구현 결과를 사용자 행동 지표와 운영 비용으로 측정하는 단계까지 확장하고자 한다.

4. AI 생성 파이프라인·검증·비용·운영 — 한의정

원문 개인보고서의 본문·표를 통합 문서 체계에 맞춰 수록하였다. 담당: 한의정

본문은 실험 ID 를 나열하지 않고 가설→검증→반례→개선의 흐름으로 설명한다. 실험 코드는 출처 확인용으로만 남기며, 비용은 OpenAI 실측 지출과 GPU 인프라 시나리오를 분리한다.

개요

본 보고서는 NVIDIA L4 GPU 한 장과 OpenAI API 한도 30 달러라는 제약에서, 공개 가중치 로컬 모델로 광고 이미지 생성의 주 경로를 구축할 수 있다는 가설을 검증한 과정이다. 목표는 모든 입력에서 상용 API 를 이기는 것이 아니라, 제품 정체성을 지키면서 광고로 사용할 수 있는 품질을 만들고 반복적인 무드 프리셋 요청의 이미지 API 호출을 제거하는 것이었다.

초기 결과에서는 제품 형상 재구성, 재료 질감 왜곡과 불필요한 소품이 반복됐다. 이를 실패 유형으로 나누고 보존 앵커, 대상별 물성 어휘, 위험 경로 배제와 후처리를 순차 적용했다. 개발 입력에서 개선한 뒤 처음 보는 11 개 이미지로 다시 확인한 결과, API 가 더 자주 선택됐지만 로컬도 향수·케이크·크루아상 등 여러 사례에서 인간 평가상 API 보다 우세했다. 따라서 전체 동급을 주장하지 않고, 로컬이 경쟁력을 보이는 조건과 남은 반례를 함께 제시한다.

팀 대시보드 기록상 OpenAI 총사용액은 20.81 달러, 이미지 API 관련 지출은 12.58 달러였다. 프리셋 local 경로는 요청당 이미지 API 과금을 발생시키지 않았지만, 이것이 자원 비용 0 을 뜻하지는 않는다. GPU 실제 청구서와 팀 전체 요청 원장이 동결되지 않아 프로젝트의 local 장당 실비와 총 회피액은 계산하지 않았다. 대신 코드의 L4 가정 0.71 달러/시간을 운영 시나리오로만 제시하고, 프로젝트 기간의 현금 지출과 향후 상용 서비스의 GPU TCO 를 분리해 해석한다.

표 1. 핵심 결과 요약

항목	관찰 결과	해석 수준
핵심 가설	한 장의 L4 에서 광고용 로컬 생성 경로 구현	구현 확인
시각 품질	API 가 더 자주 선택됐지만 local 도 개발 3 건·미사용 입력 4 건 우세	조건부 경쟁력
API 예산 격리	프리셋 local 경로의 이미지 API 호출 0	운영 경로 확인
비용 근거	OpenAI \$20.81 실측 기록 / GPU \$0.71·h ⁻¹ 가정	실측과 시나리오 분리
상용성	주 로컬 가중치 비상업 라이선스	상용 전환 전 교체 필요

출처: 개발 입력 비교·미사용 이미지 검증의 육안 판정표, 팀 OpenAI 사용량 대시보드 기록, GPU 비용 환산 코드, 모델 라이선스 고지.

1. 문제 정의와 연구 질문

광고 이미지 생성은 일반적인 미적 생성과 다른 기준을 가진다. 결과가 아름답더라도 제품의 라벨, 형상, 재료, 개수 또는 조리 상태가 입력과 다르면 광고로 사용할 수 없다. 따라서 AdNova의 핵심 문제는 '더 보기 좋은 이미지'를 만드는 데 그치지 않고, 실물 정체성을 유지하면서 연출 품질을 높이는 것이다.

프로젝트에는 단일 L4 GPU, 16GB 시스템 메모리, 100GB 이하 저장 공간, OpenAI API 한도 30 달러, 목표 응답시간 20 초라는 제약이 있었다. 특히 API 한도를 넘으면 추가 결제가 아니라 키 비활성화로 이어지므로 예산 문제는 가용성 문제이기도 했다. 이미지 생성처럼 단가가 높은 호출을 주 경로에서 격리하고, 텍스트·비전 분석만 제한적으로 사용하는 구조가 필요했다.

본 보고서는 다음 질문에 답한다.

1. 무드 프리셋 생성 경로에서 상용 이미지 API 호출을 제거할 수 있는가?
2. 로컬 경로는 상용 이미지 API 대비 어떤 품질·지연·비용 특성을 보이는가?
3. 입력 특성에 따라 엔진을 선택하는 하이브리드 정책은 인간 판정과 일치하는가?
4. 실패 분석과 프롬프트 개선을 재사용 가능한 자산으로 축적할 수 있는가?
5. 현재 구조는 기술적·법적으로 상용 서비스 전환 조건을 충족하는가?

현재 결론 로컬 경로는 프리셋 이미지 API 호출을 제거했고, 처음 보는 이미지에서도 일부 제품군은 API와 동급 이상의 광고 활용성을 보였다. 다만 모든 입력의 품질 동급과 상용 배포는 아직 입증되지 않았다.

표 2. 프로젝트 하드 제약과 설계 반응

제약	조건	설계 반응
연산	NVIDIA L4 1 장, 시스템 RAM 16GB	4 비트 양자화, 상주 워커, 모델 동시 적재 제한
저장 공간	VM 디스크 100GB 이하	컴포넌트 단위 수량, 실험별 캐시 정리
API 예산	팀 총한도 \$30, 초과 시 키 비활성화	프리셋 이미지는 로컬, API는 자유 입력·템플릿에 격리
응답 시간	목표 20 초	warm worker와 단계별 계측; 최종 약 78 초로 미달
정직성	실물과 다른 광고 금지	형상·라벨·재료 보존 가드, 입력 조건 제한

출처: 프로젝트 요구사항, 실험로그 v4, generation_service-kontext_service 설계 주석.

2. 시스템 아키텍처와 실행 경로

AdNova의 운영 정본은 최신 servingtype 계열이다. 하나의 POST /api/ads/generate 엔드포인트가 입력을 받은 뒤 template_id:style_text:style 값에 따라 세 생성 경로로 나뉜다. 웹 백엔드는 입력 검증·분석·API 호출·저장과 CPU 조판을 맡고, GENERATION_SERVICE_URL이 설정된 배포에서는 FLUX 추론만 GPU 워커(8100)로 위임한다.

그림 1. AdNova 운영 생성 구조. 무드 프리셋, 자유 지시문, 카탈로그 템플릿은 서로 다른 픽셀 엔진 정책을 사용하고 공통 후단으로 수렴한다.

2.1 요청 진입과 라우팅

업로드 이미지는 바이트 형식과 확장자를 함께 검사하고 EXIF 방향을 보정한 뒤 장변 2048 이하로 정규화한다. 분석 모델은 gpt-5.4-mini로 고정돼 있으며, 기본 설정에서는 사진-상품명 불일치 게이트와 상품명 분석·캐시를 조합한다. UNIFIED_ANALYSIS가 켜진 경우에만 사진과 상품명을 한 번의 Vision 호출로 통합 분석한다.

도메인 결정은 사진의 container_kind를 1순위, LLM이 반환한 serving_type을 2순위, 기존 어휘 규칙을 마지막 폴백으로 사용한다. 스타일 추천 API는 생성 내부 자동 라우터가 아니라 사용자가 생성 전에 호출하는 별도 선행 기능이다.

2.2 단계별 모델·실행 위치

표 3. 운영 파이프라인의 단계별 모델과 책임

단계	모델·라이브러리	실행 위치	적용 범위와 책임
입력 검증	Pillow · pillow-heif	웹 CPU	형식 검증, EXIF 보정, 장변 2048 축소, 재인코딩
분석·라우팅	gpt-5.4-mini Vision/Text + 규칙	웹 ↔ OpenAI	입력 gate, 상품 특성, container_kind-serving_type 결정
무드 프리셋	FLUX.1 Kontext [dev] Q4 GGUF + T5 NF4	GPU worker	반복 프리셋의 기본 픽셀 생성; 이미지 API 호출 없음
자유 지시문	gpt-5.4-mini 번역 → gpt-image-2 edit	웹 ↔ OpenAI	API가 기본; 실패·예산 초과·local 정책이면 Kontext 폴백

카탈로그 템플릿	gpt-image-2 edit + 결정론 crop	웹 ↔ OpenAI	template_id 전용; GPU 위임 판정보다 먼저 분기
조건부 분할·합성	BiRefNet-general rembg/ONNX	생성 프로세스	scene compose 적합 입력과 타이포 z-order 마스크에만 사용
조판·포맷	Pillow + NumPy	CPU	타이포 ON/OFF, crop/fit, 배너·카드뉴스·상세페이지
카피 gate	BLIP-base + gpt-5.4-mini LangGraph	웹 CPU ↔ OpenAI	카피 생성, 길이·금칙어 검증, 최대 3 회 재생성
저장·관측	DB·파일 · RunLogger Langfuse(선택)	웹/공통	결과·이력 저장, 시간·VRAM·토큰·비용 기록

출처: servingtype 운영 정본 commit abadad9 의 ads.py, generation_service.py, kontext_service.py, template_generation.py, copy_graph.py, scene_service.py, typography_system.py, RunLogger. localmodel 은 이전 조상이고 Qwen3-VL 이 있는 vlm001 은 미병합 실험 브랜치다.

2.3 세 생성 경로의 차이

무드 프리셋 — style 은 있고 style_text 가 없을 때 FLUX.1 Kontext 가 기본 픽셀 엔진이다. gpt-5.4-mini 분석·카피 호출은 남지만 gpt-image 계열 호출은 발생하지 않는다.

자유 지시문 — style_text 를 gpt-5.4-mini 로 영문화한 뒤 gpt-image-2 edit(quality=medium)에 전달한다.

CUSTOM_STYLE_ENGINE=local 이거나 API 실패·예산 초과 시에만 Kontext 로 폴백한다.

카탈로그 템플릿 — template_id 가 있으면 GPU 위임 판단보다 먼저 별도 경로로 빠진다. 서버 템플릿 프롬프트와 identity 보존 규칙을 gpt-image-2 에 적용하고 필요한 템플릿만 결정론적 crop 을 수행한다.

여기서 6 개 무드 프리셋과 50 개 카탈로그 template_id 는 이름만 비슷할 뿐 서로 다른 계약이다. 또한 카드뉴스·상세페이지는 worker 가 hero·top view·texture close-up·side profile·lifestyle 구도를 생성하고, 배너는 기존 clean hero 를 새로 생성하지 않고 규격별로 조판한다.

2.4 공통 후단과 운영 한계

BiRefNet 은 모든 생성의 선행 누끼가 아니다. 일반 프리셋은 원본을 Kontext 에 직접 전달하며, BiRefNet 은 조건을 통과한 장면 합성과 타이포 레이어의 z-order 마스크에만 사용한다. Pillow·NumPy 계층이 타이포·크롭·규격 조판을 결정론적으로 처리한다. LangGraph 역시 전체 이미지 파이프라인 오케스트레이터가 아니라 카피 길이·금칙어 검사와 재생성 gate 에 한정된다.

현재 GPU 직렬화는 영속 작업 큐가 아니라 프로세스 내부 락에 가깝다. 단일 GPU 시연에는 맞지만 재시작 후 대기 작업 복구, 다중 워커 분산, 요청 역등성, 공유 예산 원장은 없다. 상용 서비스에는 작업 ID·영속 큐·상태 폴링·취소·재시도 계약이 추가돼야 한다.

3. 자원 제약과 모델 선택

주 로컬 픽셀 편집기는 FLUX.1 Kontext [dev]이며 트랜스포머를 GGUF Q4_K_M 으로 양자화해 단일 L4 에 적재했다. 다만 현재 서비스가 전 구간 로컬인 것은 아니다. 무드 프리셋의 픽셀 생성만 Kontext 가 기본이고, 분석·라우팅·카피는 gpt-5.4-mini, 자유 지시문과 카탈로그 템플릿의 픽셀 생성은 기본적으로 gpt-image-2 를 사용한다. 상주 워커 기록은 약 14.2GiB, 쿨드 요청 단일 피크는 약 13.25GiB 였으며 패키지·드라이버·모델 revision 이 고정되지 않아 장기 재현성에는 한계가 있다.

표 4. 모델 및 오케스트레이션 구성

구성	실제 역할	운영 판단	주요 제한
FLUX.1 Kontext [dev]	무드 프리셋 픽셀 편집	주 로컬 엔진	비상업 라이선스
gpt-image-2 edit	자유 지시·카탈로그 템플릿	상용 API 엔진·기준선	호출 비용·seed 미지원
gpt-5.4-mini	Vision/Text 분석·라우팅·카피·번역	현재 운영 모델	OpenAI 한도 의존
BLIP-base	기본 이미지 캡션	로컬 CPU 보조	세부 제품 이해 한계
BiRefNet-general	조건부 장면 합성·타이포 마스크	선택적 보존 도구	보편 선행 누끼 아님
Pillow + NumPy	타이포·crop/fit·포맷 조판	결정론적 CPU 후처리	생성적 연출은 하지 않음
LangGraph · RunLogger · Langfuse	카피 gate · 실행 원장 · 선택 추적	필요 구간에만 사용	전량 추적·영속 큐 아님
Qwen3-VL 2B-4B	라우팅·카피·묘사 실험	미병합·미채택	운영 경로에 없음

출처: servingtype 운영 정본의 kontext_service.py, generation_service.py, gpt_service.py, copy_graph.py, scene_service.py, typography_system.py, RunLogger 와 미병합 vlm001 실험 감사.

3.1 파인튜닝을 수행하지 않은 이유

LoRA 학습보다 먼저 생성 결과에서 입력·지시문·출력 삼중항을 수확하는 프로토타입을 만들었다. 2026 년 7 월 10 일 원장 27 행을 검사했으나 심미·judge 점수가 함께 기록된 학습 가능 표본은 0 건이었다. 12 장 수준의 소규모 입력으로 스타일 LoRA 를 학습하면 과적합과 제품 정체성 붕괴 위험이 컸으므로 학습을 수행하지 않았다.

수확기는 자동 프로덕션 경로에 연결된 상태가 아니었고 이후 미사용 코드 정리에서 제거됐다. 따라서 현재 상태는 '데이터 플라이휠이 준비됐다'가 아니라 '학습 조건을 정의하고 프로토타입으로 확인했으나 운영 연결과 학습은 수행하지 않았다'이다. 본 프로젝트에서 20 일간 조정된 것은 모델 가중치가 아니라 프롬프트, 라우팅, 후처리 규칙이다.

상업성 공개 가중치와 오픈소스는 같은 말이 아니다. 주 로컬 가중치는 비상업 라이선스이므로 교육·내부 데모에는 사용할 수 있지만 상용 전환에는 별도 라이선스나 상업 허용 모델이 필요하다.

4. 가설과 개선 과정

초기 가설은 단순했다. 제품 사진과 무드 지시를 함께 주면, 단일 L4 에 올린 로컬 편집 모델도 상용 이미지 API 에 의존하지 않고 광고에 사용할 수 있는 장면을 만들 수 있다는 것이었다. 검증의 초점은 최고 심미 점수가 아니라 제품 정체성을 유지한 채 배경·조명·구도를 바꾸는 능력이었다.

4.1 첫 결과가 드러낸 문제

초기 결과는 보기 좋은 경우에도 광고 기준을 자주 어겼다. 손에 든 마우스·향수처럼 강제 SKU 는 부품과 외곽이 재구성됐고, 면류와 쿠키는 형태·개수가 달라졌다. 금지문을 길게 붙이면 해결될 것이라는 가설도 실패했다. 확산 모델에서는 금지문 속 위험 명사 자체가 조건 신호로 남아, '스파게티를 만들지 말라'는 문장이 오히려 스파게티 형태를 재호출하는 경우가 있었다.

표 5. 실패를 개선 규칙으로 바꾼 핵심 근거

관찰된 실패	원인 가설	변경	결과와 해석
금지한 소품·형태가 재등장	금지문 속 위험 명사가 조건 신호로 작동	충돌 장면 후보를 제거하고 긍정 앵커를 앞에 배치	재발 감소; '더 긴 금지문' 가설 기각
손에 든 rigid SKU 보존 실패	손·제품 경계와 자세를 동시에 재해석	손-held 장면을 버리고 tabletop 경로로 제한	0/3 → 2/3 보존; 프롬프트 천장 확인
면류 형태 왜곡	범용 미감 어휘가 음식의 물성을 덮음	가닥·굵기·국물·표면 등 물성 어휘를 도메인별로 주입	동일 local 입력·seed 에서 4/6 → 6/6
쿠키가 서로 합쳐짐	케이크용 이상화 질감과 초코 스왑 규칙 충돌	구움과자 어휘를 요리명 우선으로 분리	합침은 해소됐지만 9 개→7 개; 개수 보존 성공은 아님
소품이 상품을 가림	무드 연출과 정체성 규칙의 우선순위 충돌	라벨·접촉 소품·최종 override 를 후단에 명시	여러 사례 개선; 단회 표본이라 반복 검증 필요
object→local 단순 라우팅	상품 대분류가 형상·라벨 위험을 설명하지 못함	손·라벨·외곽 복잡도·배경제거 가능성을 특징으로 추가	홀드아웃 일치 6/11; 단순 규칙 기각

출처: KPI 실험로그의 장면 안전화·면류·구움과자 수정 기록과 홀드아웃 판정. 숫자는 성공을 과장하지 않도록 '개선된 결함'과 '남은 결함'을 분리해 읽는다.

이 과정에서 가장 중요한 판단은 '한 결함이 줄었다'와 '제품 정체성이 완전히 보존됐다'를 같은 성공으로 쓰지 않는 것이었다. 쿠키의 합침은 해결됐지만 개수는 9 개에서 7 개로 바뀌었고, 손-held rigid SKU 는 프롬프트만으로 0/3 이었다. 이 반례가 입력 제한과 라우팅 정책의 근거가 됐다.

4.2 프롬프트보다 먼저 바꾼 것

충돌하는 연출은 금지문으로 억누르지 않고 장면 후보에서 제외했다.

유지할 형상·개수·라벨·재료를 프롬프트 앞부분의 구체적인 긍정 앵커로 고정했다.

glossy 같은 범용 형용사를 마블링·크림층·면 가닥·유리 반사처럼 대상별 물성 어휘로 교체했다.

손·긴 라벨·복잡 외곽은 위험 특징으로 취급해 API·tabletop·조건부 합성 경로로 보냈다.

생성, 정체성 보존, 타이포·포맷 조판을 서로 다른 계층의 책임으로 분리했다.

결과적으로 개선의 단위는 프롬프트 한 문장이 아니라 '입력 허용 범위 + 엔진 선택 + 보존 규칙 + 결정론적 후처리'가 됐다. 이 책임 분리가 local 과 API 를 같은 광고 기준으로 비교하고, 실패가 발생한 계층을 추적할 수 있게 했다.

4.3 API 를 대체 대상이 아니라 품질 기준선으로 사용

상용 이미지 API 는 로컬을 포기하기 위한 대안이 아니라 품질 격차를 드러내는 기준선으로 사용했다. 첫 비교는 API arm 에 네 문장짜리 초안만 넣고 local 에는 누적된 보존 규칙을 적용해 공정하지 않았다. 이후 엔진과 입력은 그대로 두고, local 에서 만든 긍정 앵커·정체성 override·위험 소품 규칙을 API 지시문에 이식해 지시문 자산과 엔진 효과를 분리했다.

그림 2. 같은 상용 이미지 엔진에 local 개발 과정의 보존 규칙을 이식한 탐색 비교. 실험 코드 대신 입력·조건·변경점을 직접 표시했다.

꽃등심·음료·마우스에서 보존 또는 연출 개선 신호가 관찰됐지만, 표본은 n=3 이고 조건당 1 회 생성이다. gpt-image-2 는 seed 를 통제할 수 없으므로 프롬프트 전이의 인과 효과로 단정하지 않고 후속 반복 검증의 근거로만 사용한다.

4.4 프롬프트를 재사용 가능한 검증 자산으로 전환

모델별로 완성 문장을 복사하는 대신, 형상·라벨·개수·재료·접촉 소품·최종 override 를 체크리스트와 구조화 블록으로 관리했다. 이 자산은 local 과 API 양쪽에 적용할 수 있어 엔진을 바꾸더라도 동일한 실패 기준을 유지한다. 다만 현재 근거는 소표본 탐색이므로 '모델 간 지식 이전이 입증됐다'가 아니라 '재사용 가능한 규칙 후보를 확보했다'가 정확한 결론이다.

5. 검증 방법: 개발 입력과 미사용 이미지

검증은 모든 실험을 나열하는 방식이 아니라, 하나의 가설이 다음 설계 결정으로 이어지는지 확인하는 순서로 진행했다. 개발에 사용한 입력에서는 실패 유형과 개선 방향을 찾고, 그 규칙을 고정한 뒤 처음 보는 이미지에서 다시 확인했다.

로컬 기준선 생성 — 제품 정체성과 광고 활용성을 해치는 실패를 수집했다.

실패 기반 개선 — 프롬프트 위치, 물성 어휘, 라우팅과 후처리를 바꿨다.

API 기준선 비교 — 품질 격차와 local 이 더 강한 조건을 찾았다.

미사용 이미지 재검증 — 개발 과정에서 보지 않은 11 개 입력으로 일반화와 반례를 확인했다.

표 6. 핵심 가설과 검증 결과

가설	어떻게 확인했는가	관찰	판단	다음 개선
로컬도 광고용 품질을 만들 수 있다	개발 입력 12 개와 미사용 입력 11 개를 API 기준선과 비교	local 우세 사례 3 건·4 건	조건부 지지	강점 입력을 명시적으로 라우팅
보존 규칙이 실패를 줄인다	형상·재료·소품 결함의 전후 사례 확인	여러 사례에서 복원, 일부 재발	부분 지지	반복 생성·정체성 지표 추가
object→local 규칙이면 충분하다	미사용 11 개에서 정책 선택과 인간 선호 비교	6/11 일치	기각	라벨·손·형상·배경제거 가능성 사용
local 은 이미지 API 예산을 줄인다	프리셋 생성 경로의 실제 호출·원장 확인	이미지 API 호출 0	운영상 지지	GPU TCO 와 호출 절감을 분리 보고

출처: 개발 입력 3 경로 비교(HYB-001), 미사용 이미지 검증(HOLDOUT-001), 호출 원장과 육안 판정표. 코드는 근거 색인에서만 사용한다.

5.1 평가 기준과 결론의 범위

주 평가는 결과 출처를 가린 상태의 아트디렉터 육안 판정으로 두고, LLM judge 는 보조 지표로만 사용했다.

광고용 이미지는 심미성뿐 아니라 제품 형상, 재료, 라벨, 개수와 불필요한 추가 요소를 함께 봐야 하기 때문이다.

API 출력은 seed 를 지원하지 않고 각 조건의 반복 수도 부족하므로, 결과는 평균 성능의 통계적 동등성보다 강점·실패 조건을 찾는 탐색적 근거로 해석한다.

원장 한계 실행 로거는 요청별 시간과 추정비용을 기록하지만, 팀원 전체의 생성·재생성·실패를 포함한 하나의 불변 원장은 보존되지 않았다. 따라서 총 생성 건수나 프로젝트 전체 local 장당 실비를 역산하지 않는다.

6. 검증 결과와 설계 변화

가장 중요한 결과는 'local 이 항상 API 보다 낫다'가 아니다. API 가 전체적으로 더 자주 선택되는 상황에서도, local 이 제품 정체성과 클린 재연출에서 분명한 경쟁 구간을 만들었고 그 조건을 시스템 설계에 반영할 수 있게 됐다는 점이다.

6.1 전체 결과: 평균과 경쟁 구간을 함께 본다

표 7. 두 단계 인간 평가 요약

검증 단계	API 우세	local 우세	동률·공통 실패	무엇을 배웠는가
개발 입력 12 개	7	3	동률 1 · 공통 실패 1	API 품질 기준과 local 강점 후보 확인
처음 보는 입력 11 개	7	4	—	향수·케이크·크루아상에 서 local 경쟁력 재확인

개발 입력과 미사용 입력은 목적이 다르므로 23 건을 하나의 승률로 합치지 않는다. 개발 입력은 개선에 사용됐고, 미사용 입력은 개선 규칙의 반례를 찾는 데 사용됐다.

API 는 복합 음식 장면과 자유로운 재연출에서 더 안정적이었다. 반면 local 은 배경을 단순화해 상품을 전면에 놓는 클린 광고 컷과, 원본의 형상·층 구조를 유지해야 하는 일부 제품에서 더 좋은 평가를 받았다. 이 결과는 local 의 품질을 '저가 대체재'가 아니라 특정 조건에서 우선 선택할 수 있는 엔진으로 해석하게 했다.

6.2 시각 결과: 처음 보는 이미지에서도 확인된 local 의 경쟁력

아래 사례는 프롬프트 조정에 사용하지 않은 입력이다. 원본을 함께 제시해 배경 연출뿐 아니라 제품 형상·재료·소품 보존을 직접 비교한다. 케이크와 크루아상은 local 의 경쟁 구간을, 향수는 보존과 연출의 trade-off 를, 헤드폰은 API 가 명확히 우세한 반례를 보여준다.

사례A — 딸기 생크림 케이크

그림 3. local 은 인물·휴대폰·카페 잡음을 제거하고 케이크의 층과 딸기를 유지한 제품 중심 광고 컷을 만들었다.

사례B — 크루아상

그림 4. local 은 제품과 식기의 정체성을 지키면서 배경 소품을 단순화해 광고 초점을 명확히 했다.

사례c — 향수

그림 5. 향수는 local 의 clean cutout 보존과 API 의 스튜디오 연출이 다른 장점을 보였다. 한 축의 평균 점수보다 사용 목적별 라우팅이 필요함을 보여준다.

반례 — 헤드폰

그림 6. 헤드폰에서는 API 가 제품 형상을 유지하면서 광고 조명·반사·구도를 함께 완성했다. '사물은 local' 규칙이 성립하지 않는 직접 반례다.

출처: HOLDOUT001_육안판정_20260721. 조건당 1 회 생성·단일 아트디렉터 판정이며 API seed 는 통제되지 않았다. 따라서 이 그림은 통계적 동급이 아니라 경쟁 구간과 반례를 시각적으로 확인하는 근거다.

6.3 반례가 바꾼 하이브리드 설계

초기 정책은 사물은 local, 나머지는 API 로 보내는 단순 규칙이었다. 처음 보는 입력에서 이 정책과 인간 선호가 일치한 것은 11 건 중 6 건뿐이었다. 텀블러와 헤드폰은 사물이지만 API 가 우세했고, 케이크와 크루아상은 비사물이지만 local 이 우세했다. 따라서 상품 대분류만으로 엔진을 고르는 가설은 기각했다.

라벨·글자·손 포함 여부처럼 정체성 훼손 위험을 입력 특징으로 사용한다.

배경 제거가 쉬운지, 상품 외곽과 재질이 얼마나 복잡한지 함께 본다.

고위험·자유 지시문은 API, 반복 프리셋과 클린 상품 컷은 local 후보로 둔다.

정책은 새 홀드아웃에서 다시 검증하며 엔진별 반복 생성 분산을 함께 측정한다.

7. 비용 구조와 예산 효과

비용은 하나의 '장당 단가'로 합치지 않는다. 프로젝트 기간에 실제로 현금이 지출됐는지 보는 회계와, 향후 상용 서비스가 API-only·local·hybrid 중 어떤 구조를 선택할지 비교하는 TCO 는 질문이 다르다. local 의 이미지 API 항목이 0 이라는 사실과 전체 자원 비용이 0 이라는 주장을 분리한다.

7.1 응답 시간과 기존 장당 환산의 한계

local 요청의 p50 은 약 69~70 초였고 API 는 검증 구간에 따라 약 24~38 초였다. 기존 원장은 요청 벽시계에 0.71 달러/시간을 곱해 '활성시간 귀속 원가'를 만들었지만, 이는 GPU 가 100% 활용되거나 생성 시간만 초 단위로 과금된다는 가정에 의존한다. 상주 VM 의 유휴시간과 CPU·RAM·디스크·네트워크 비용을 포함하지 않으므로 장당 실비로 제시하지 않는다.

표 8. 응답 시간 비교

검증 구간	local p50	API p50	해석
-------	-----------	---------	----

개발 입력	69.02 초	37.83 초	개선 과정에 사용된 입력
처음 보는 입력	69.96 초	23.90 초	입력 해상도·실행 시각 차이 포함

비용 표에서는 이 시간을 장당 실비로 환산하지 않고 처리량·용량 계획 지표로만 사용한다.

7.2 먼저 구분해야 할 두 회계 기준

표 9. 프로젝트 현금 지출과 향후 상용 TCO 의 차이

회계 기준	묻는 질문	GPU 해석	현재 보고 가능한 것
프로젝트 기간 현금 지출	실험 기간에 실제로 얼마가 결제됐는가?	선불 크레딧·이미 계약된 VM 이면 해당 기간의 추가 현금 지출이 0 일 수 있음	OpenAI 대시보드 \$20.81; GPU 청구서 부재로 총 현금원가 미산출
단기 라우팅 의사결정	이미 컨 GPU 에서 다음 요청을 local/API 중 어디로 보낼까?	해당 계약 기간에는 공통 고정비일 수 있음; local 요청은 이미지 API 과금을 추가하지 않음	증분비용 비교 가능, 단 가용 GPU 용량의 기회비용은 남음
향후 상용 서비스 TCO	API-only 라면 GPU 를 제거할 수 있는가?	제거 가능하면 GPU 는 회피 가능한 비용이지 매몰비용이 아님	$R_{actual} - H_{billed}$ ·가동률·CPU/RAM/disk/egress/monitoring 필요

정리하면 '어차피 서버에 GPU 가 필요하다'는 전제가 계약 기간 동안 참이고 local 때문에 추가 용량이 필요하지 않을 때만 GPU 를 단기 공통 고정비로 볼 수 있다. 반대로 API-only 서비스가 GPU 를 제거할 수 있다면 향후 GPU 비용은 매몰비용이 아니라 선택 가능한 비용이다.

7.3 GPU 비용 시나리오와 운영식

현재 배포는 모델을 미리 적재한 상주 GPU 워커였다. 따라서 실제 GPU 비용은 이미지 수가 아니라 GPU 가 포함된 VM 의 청구 시간으로 계산해야 한다. 아래 0.71 달러/시간은 코드의 L4 기본 가정이며 실제 공급자·리전·머신 구성과 청구서로 교체해야 한다.

표 10. GPU 비용 시나리오

항목	계산	금액·의미
코드 기본 가정	$R = \$0.71/h$	실청구가 아닌 L4 근사값
1 일 상주	$\$0.71 \times 24h$	\$17.04/일

30 일 상주	$\$0.71 \times 720h$	\$511.20/월
실제 월 GPU 비용	$R_{actual} \times H_{billed}$	선택 GPU 와 실제 가동시간을 입력
추가 TCO	CPU-RAM-disk-egress-monitoring	청구서 기반 비용을 별도 합산

팀 전체 생성 건수와 GPU uptime 이 같은 원장으로 동결되지 않았으므로 프로젝트 전체 local 비용과 실제 local 장당 단가는 산출하지 않는다.

표 11. 운영 방식별 올바른 비용식

운영 방식	GPU	이미지 API	총비용 식
API-only	제거 가능	모든 이미지 요청	$C_{web} + C_{text} + \sum C_{image_api,i}$
local 중심	$R_{actual} \times H_{billed}$	프리셋 0 회	$C_{web} + C_{text} + R_{actual} \times H_{billed} + C_{storage/egress}$
hybrid	$R_{actual} \times H_{billed}$	API 로 보낸 요청만	$C_{web} + C_{text} + R_{actual} \times H_{billed} + \sum (i \in API) C_{image_api,i}$
이미 확보한 GPU 의 단기 선택	공통 고정비 가능	호출 시만 증분	local 은 이미지 API 과금을 추가하지 않지만 용량 기회비용은 남음

$C_{image_api,i}$ 는 각 요청의 실제 청구액이다. 품질·크기·모델별 단가가 달라질 수 있으므로 하나의 고정 장당 단가로 치환하지 않는다.

7.4 실측 OpenAI 지출과 local 의 예산 효과

표 12. OpenAI 예산 현황

항목	금액	의미
팀 전체 OpenAI 사용액	\$20.81 / \$30.00	대시보드 기록, 한도의 69.4%
이미지 API 관련 지출	\$12.58	확인된 OpenAI 지출의 큰 비중
잔여 한도	\$9.19	추가 실험·운영에 사용 가능한 범위
프리셋 local 이미지 호출	\$0 image API/request	픽셀 생성의 이미지 API 비용을 발생시키지 않음

잠재 회피액

미산출

Σ(각 local 요청을 API 로 처리했을 때의
실제 청구액); 요청 원장·청구 export 부재

출처: 2026-07-28 팀 사용량 대시보드 수기 기록. 청구 export 와 팀 전체 요청 원장이 보존되지 않아 회피액을
숫자로 환산하지 않는다.

비용 결론 프리셋 local 경로는 요청당 이미지 API 과금을 발생시키지 않아 외부 API 예산 소비를 API 로 라우팅한
요청에만 한정한다. 다만 전체 TCO 가 더 낮은지는 GPU 단가·청구시간·가동률·요청량을 포함한 별도 운영
시나리오에서 결정된다.

8. 보조 트랙: VLM 과 RAG 고객지원

8.1 로컬 VLM 자가호스팅

Qwen3-VL 2B-4B 를 사용해 상품 라우팅, 이미지 묘사와 SNS 카피를 OpenAI 없이 수행할 수 있는지 확인했다. 2B
라우팅은 6/12, 4B 는 10/12 였고, 실제 운영 광고 4 건의 카피 비교에서는 무-AI 템플릿보다 나았지만 GPT 보다
낮았다. 이미지 묘사도 전체 맥락은 파악했으나 브랜드·색·소품 오류가 남았다.

표 13. 로컬 VLM 채택 여부 판단

평가	관찰 결과	최종 판단
2B 상품 라우팅	6/12, 카페 0/4·한국어 오역	부적합
4B 상품 라우팅	10/12	개선됐으나 GPT 보다 낮음
실제 광고 카피 4 건	무-AI < VLM 4B < GPT	운영 카피는 GPT 유지
이미지 묘사 4 건	맥락은 파악, 세부 오류	저비용 요약 후보
브랜드 상태	실험 브랜치에만 존재	최종 서비스에서 제거

출처: feature/vlm-001 러너·매니페스트와 KPI 94~99 행. 원시 결과 JSON/MD 는 최종 브랜치에 보존되지 않았다.

이 트랙의 성과는 자가호스팅 채택이 아니라 미채택 근거를 계측한 것이다. 개인정보 보호나 온프레미스가
품질보다 중요한 환경에서는 저신뢰 스크리닝으로 재검토할 수 있지만, 현재 프로덕션 경로에는 포함하지 않는다.

8.2 RAG 고객지원 챗봇

고객지원 챗봇은 FAQ YAML 을 단일 지식 원본으로 사용하고, BM25·한국어 형태소·자모 오타 보정·저신뢰 query
rewrite 를 결합했다. 검색 신뢰도가 충분하면 FAQ 근거 답변을 생성하고, 낮으면 재작성 후 다시 검색한다. 그래도
실패하면 일반 사용법 질문에 제한적으로 답하는 tier-2 또는 1:1 문의 에스컬레이션으로 이동한다.

표 14. 고객지원 챗봇의 질문별 검증

검증 내용	표본/범위	결과	해석
기본 검색·생성 경로	골든 48	BM25 hit@1 91.67%	베이스라인
dense 검색 보강	dense 추가	hit@1 94.44%	메모리 대비 이득 작아 기본 off
저신뢰 질문 재작성	query rewrite	hit@3 100%, precision 100%	채택
지식베이스 확장	KB 34-guide 10	hit@1 95.65%, guide 100%	검색 기준
상위 모델 judge 재평가	생성 20·상위 judge	F 4.58 / R 4.83 / reject 4.88	self-eval 5/5/5 정정

출처: 실험로그_챗봇_v1.md. off-topic n=4, injection n=2 의 100%는 소표본 내부 결과다.

생성 모델과 동일한 gpt-5.4-mini 를 judge 로 사용했을 때 5.0/5.0/5.0 이었지만, 상위 티어 gpt-5.4 로 같은 20 건을 재평가하자 4.583/4.833/4.875 로 정정됐다. 이는 judge 와 생성 모델을 분리해야 한다는 원칙을 정량적으로 보여준다. 다만 같은 OpenAI 계열이라는 벤더 편향은 남는다.

챗봇 안전 한계 현재 근거 게이트는 허용된 FAQ ID 문자열이 답변에 하나 포함됐는지 확인할 뿐 의미적 함의는 검증하지 않는다. 기본 활성화된 tier-2 일반답변의 요금·정책·계정 누출률도 전용 골든셋으로 아직 정량화하지 않았다.

9. 운영·재현성·라이선스

9.1 확인된 구현과 운영 한계

표 15. 구현 상태와 프로덕션 준비도

항목	확인된 구현	남은 위험	상태
웹/GPU 워커 분리	HTTP 계약·systemd 서비스	영속 큐·작업 복구 없음	부분 준비
serving_type	5 값 계약·클램프·킬스위치	container_kind 가 우선되는 경로	구현
이미지 API 폴백	자유 입력 실패 시 local	예산 카운터가 프로세스 메모리	부분 준비
실행 원장	RunLogger 스키마·Langfuse 세션	정본 snapshot·전량 커버리지 없음	부분 준비

테스트	표적 테스트 50 건 확인	선택 의존성 누락으로 전체 수집 오류	부분 검증
재현 환경	모델명·GGUF 파일 지정	AI 패키지·model revision 미고정	미완료

출처: 최종 develop 계열 코드, servingtype/localmodel/vlm001 작업 트리 감사.

9.2 재현 패키지

보고서 기준일의 운영 runs.jsonl 읽기 전용 snapshot 과 SHA-256

테스트·운영 실행 분리 규칙과 집계 스크립트

입력 이미지 해시, Git SHA, CUDA·드라이버·GPU 환경 ID

Python lockfile 과 Hugging Face model revision/commit hash

동일 기준 시각의 OpenAI 청구 export 와 Langfuse export

인간 판정 원장, 평가자 수, 블라인드 순서와 tie 처리 규칙

9.3 모델 및 이미지 라이선스

주 편집 가중치는 비상업 라이선스여서 현재 결과는 기술 데모 범위에 머문다. 상용 전환 전 상업 이용 가능한 모델이나 API 로 교체해 동일 홀드아웃을 재실행해야 한다. HOLDOUT 의 CC BY·CC BY-SA 입력도 저작자·원본 URL·라이선스 버전·변형 여부를 보완해야 하며, 현재 출처 파일만으로는 외부 공개 요건을 충족하지 못한다.

10. 결론과 다음 검증 계획

AdNova 는 한 장의 L4 와 제한된 API 예산에서 로컬 광고 이미지 생성 경로를 실제 서비스 구조로 만들고, 실패를 프롬프트·라우팅·후처리 문제로 분해해 개선했다. API 가 전체적으로 더 자주 선택됐지만, 처음 보는 케이크와 크루아상 등에서 local 이 인간 평가상 더 높은 광고 활용성을 보였다. 이는 로컬 모델이 단순한 저가 대체가 아니라 조건에 따라 선택할 수 있는 품질 엔진이 되었음을 보여준다.

10.1 확인된 것

무드 프리셋의 픽셀 생성은 gpt-image 계열 호출 없이 FLUX.1 Kontext 로 동작한다. 다만 분석·라우팅·카피에는 gpt-5.4-mini API 가 남는다.

웹/API 와 GPU 워커, 프리셋과 자유 입력 경로가 분리돼 있다.

상용 이미지 API 가 전체적으로 더 자주 선택됐지만 local 도 개발 입력 3 건과 미사용 입력 4 건에서 우세했다.

하이브리드의 비용·속도는 라우팅 비중에 따라 크게 달라진다.

VLM·Best-of-N·LoRA 등 품질 또는 데이터 조건을 충족하지 못한 트랙은 미채택으로 남았다.

10.2 관찰됐지만 아직 입증되지 않은 것

제품 중심 clean cut 과 일부 디저트에서 local 이 API 와 동급 이상의 광고 활용성을 가질 가능성

local 에서 축적한 보존 규칙이 API 결과에도 도움이 될 가능성 — n=3 단회·API seed 미통제로 탐색적 관찰에 한정

이미 계약된 GPU 에서 local 요청이 이미지 API 과금을 추가로 발생시키지 않는 단기 예산 보호 효과

도메인·형상·라벨·손·배경제거 가능성을 조합한 세밀한 하이브리드 정책의 가능성

최종 판단 핵심 성과는 한 장의 L4 에서 광고에 사용할 수 있는 local 프리셋 경로를 구현하고, API 와 경쟁할 수 있는 품질 구간을 원본-LOCAL-API 로 확인했으며, 반복 프리셋의 이미지 API 지출을 격리한 것이다. 헤드폰 같은 API 우세 반례와 비용의 두 회계 기준은 local 의 장점을 약화하는 것이 아니라 언제 어떤 엔진을 선택해야 하는지 더 정확하게 만든다.

10.3 다음 검증 우선순위

6. 각 입력·엔진을 최소 5~10 회 반복하고 생성 순서를 무작위화한다.
7. 다수 평가자의 블라인드 쌍대 비교와 평가자 간 일치도를 기록한다.
8. 음식과 사물에 각각 맞는 품질·정체성 루브릭을 사용한다.
9. object 하나가 아니라 구도, 손 포함 여부, 라벨, 형상 복잡도와 배경 제거 가능성을 정책 특징으로 사용한다.
10. 운영 원장·청구 export·모델 revision 을 동결해 독립 재계산 가능한 증거 패키지를 만든다.
11. 상업 이용 가능한 모델로 교체한 뒤 동일 홀드아웃 벤치마크를 재실행한다.
12. 실제 GPU 청구서·가동시간과 팀 전체 요청 원장을 동결해 월 TCO 와 회피된 이미지 API 지출을 계산한다.

부록 A. 검증 자료 색인

표 A-1. 핵심 근거 파일

ID	근거	용도
A1	~/Desktop/AdNova/KPI_실험로그정리_20260717.md	전 기간 실험 목록·대시보드
A2	~/ai-ad-generator-AdNova-rule/실험로그_이미지생성_v4.md	이미지 실험 정보 회고 로그
A3	~/ai-ad-generator-AdNova-rule/실험로그_챗봇_v1.md	RAG 실험 정보 로그

A4	~/Desktop/AdNova/HYB001_results_20260721/	개발셋 3 암 요약·judge·산출물
A5	~/Desktop/AdNova/HOLDOUT001_results_20260721/	홀드아웃 요약·rows JSONL·산출물
A6	~/Desktop/AdNova/보고서_그림_20260728/	APIQ·HYB·HOLDOUT 접촉시트
A7	GitHub elena89y/ai-ad-generator-AdNova	최종 코드 계열; 감사 기준 SHA abadad9
A8	feature/local-model-track-feature/vlm-001	미채택·실험 전용 구현 확인

로컬 경로는 보고서 작성 환경 기준이다. 배포 가능한 재현 패키지에는 snapshot·hash·집계 스크립트를 별도 포함해야 한다.

부록 B. 타당성 위협과 결론 영향

표 B-1. 타당성 위협

한계	결론에 미치는 영향	보완 방법
API seed 미통제	엔진 효과와 확률 변동 분리 불가	반복 생성과 승률·분산 보고
표본 11~12 건	상품군 일반화 불가	계층화된 신규 홀드아웃 확대
단일 아트디렉터 판정	평가자 편향과 일치도 미측정	다수 평가자·블라인드·tie 규칙
혼합 도메인 LLM judge	사물 점수의 의미 왜곡	도메인별 루브릭과 인간 보조
추정 단가	장당 비용 우위 불확실	청구 export·실시간 원가 측정
정본 원장 부재	일부 총계 독립 재계산 불가	snapshot·hash·집계 스크립트
비상업 가중치	상용 배포 결론 제한	상업 허용 모델 교체 후 재검증
출처 메타데이터 불완전	외부 공개 시 attribution 위험	저작자·URL·라이선스·변형 기록

이 표의 한계는 결론의 무효화를 의미하지 않는다. 각 결론이 어느 수준까지 말할 수 있는지를 제한한다.

부록 C. HOLDOUT 입력 출처와 라이선스

홀드아웃 입력은 Wikimedia Commons 에서 수집한 것으로 기록되어 있다. 아래 표는 프로젝트 출처 파일의 내용을 옮긴 것이다. CC BY·CC BY-SA 자료는 외부 공개 전에 저작자와 원본 URL, 라이선스 링크, 변형 여부를 추가해야 한다. 현재 로컬 기록만으로는 완전한 attribution 이 아니다.

표 C-1. HOLDOUT-001 입력 출처 기록

입력	Commons 파일명	라이선스
김치찌개	Kimchi_jjigae_-_Kogi_Brighton_2023-11-15.jpg	CC0
한정식상차림	0606_hanjeongsik_damyang.jpg	Public domain
떡볶이	Tteokbokki_in_Handan_1.jpg	CC BY-SA 4.0
까르보나라	Pasta_Carbonara_at_restaurant_Vltava.jpg	CC BY-SA 4.0
딸기롤케이크	Fresh_strawberry_roll_cake_-_Ikigai_Coffee_2024-03-31.jpg	CC0
딸기생크림케이크	Strawberry_cream_cake_-_Rawlab_2025-08-31.jpg	CC0
크루아상	Croissant_and_espresso_at_Signora.jpg	CC BY-SA 4.0
핑크레모네이드	Pink_lemonade_in_personalized_mason_jar_glass_mugs...	CC BY 2.0
향수	Armani_perfume_1.7oz.JPG	CC BY-SA 3.0
헤드폰	Afterglow_AGU.1_Wireless_Headphones.jpg	CC BY-SA 3.0
텀블러	AquaFlask.jpg	CC BY-SA 4.0

출처: ~/Desktop/AdNova/HOLDOUT_inputs/출처_라이선스.md. 외부 배포 전 attribution 보완 필요.

5. 백엔드·DB·관리자·DevOps — 김범수

원문 개인보고서의 본문·표를 통합 문서 체계에 맞춰 수록하였다. 담당: 김범수

1. 프로젝트 개요와 담당 역할

AdNova는 소상공인이 상품 사진을 활용해 광고 콘텐츠를 만들 수 있도록 기획한 서비스다. 사용자는 상품 이미지를 올리고, 원하는 스타일이나 요청 사항을 입력한 뒤 광고 이미지와 문구를 확인한다. 이후 생성된 결과는 내 광고 목록에서 다시 볼 수 있고, 필요한 경우 삭제하거나 다운로드할 수 있다. 프로젝트의 핵심은 생성형 AI를 활용하는 것이지만, 최종 결과물은 사용자가 실제로 접속해 사용할 수 있는 웹 서비스였다.

내 담당 역할은 백엔드 및 DevOps였다. 백엔드 쪽에서는 DB 구조, 이미지 업로드, 대시보드, 광고 결과 저장, 히스토리, 계정 관리, 구독과 크레딧, 관리자 API를 주로 맡았다. DevOps 쪽에서는 GCP VM에서 서버를 실행하고, systemd와 Docker Compose 기반 실행 구조를 확인하며, 최종적으로 Next.js와 FastAPI 일반 웹 서비스를 Docker로 전환하는 작업을 진행했다.

보고서에서 주의한 점은 다른 팀원의 작업과 내 작업을 섞지 않는 것이다. AI 생성 모델이나 프롬프트 파이프라인 자체는 내 담당으로 적지 않았다. 로그인과 회원가입의 초기 기능도 다른 팀원 작업이 포함되어 있어, 내가 직접 다룬 부분은 DB 정합성 확인, 중복 처리, 이메일 인증 임시 모드, 세션과 배포 흐름 보완처럼 실제로 수정하거나 검증한 범위로 한정했다.

1.1 담당 범위 요약

구분	내가 진행한 내용
DB 기반	SQLite 연결 설정, SQLAlchemy 모델 정의, Pydantic 스키마, CRUD, ERD 문서, 테이블 생성 검증
일반 사용자 API	이미지 업로드, 대시보드 요약, 광고 결과 저장, 히스토리 조회·삭제·다운로드, 계정·구독·크레딧 기능
관리자 기능	회원·구독·결제·문의·환불 관리 API, 관리자 역할 구분, 감사 로그, 관리자 계정 관리
운영 안정화	파일 검증, 경로 검증, SQLite 외래키 활성화, 마이그레이션 구조, 회원탈퇴 데이터 정리
DevOps	GCP VM 실행 환경 확인, systemd 서비스 확인, Docker Compose 전환, 웹 백엔드 의존성 분리, Docker 자동 배포 구조 준비

2. 백엔드 구조를 잡은 과정

2.1 DB 구조부터 시작한 이유

처음에는 실제 API 를 많이 붙이기보다 서비스에서 어떤 데이터가 남아야 하는지부터 정리했다. 회원가입, 이미지 업로드, 광고 생성 기록, 히스토리 조회가 모두 연결되려면 먼저 공통으로 사용할 DB 구조가 필요했기 때문이다. 그래서 7 월 1 일에는 User, Image, Advertisement, History 네 가지 모델을 중심으로 초기 DB 레이어를 만들었다. 초기 DB 는 SQLite 로 잡았다. 운영까지 생각하면 PostgreSQL 이 더 적합하지만, 프로젝트 초반에는 팀원이 로컬과 GCP 에서 빠르게 실행해 보는 것이 더 중요하다고 판단했다. 대신 SQLAlchemy 를 사용해 DB 접근 코드를 ORM 기반으로 작성했고, 환경변수로 DATABASE_URL 을 바꿀 수 있게 구성했다.

구조를 잡을 때는 SQLAlchemy 모델, Pydantic 스키마, CRUD 를 나눴다. SQLAlchemy 모델은 실제 테이블과 관계를 담당하고, Pydantic 스키마는 API 요청과 응답 형식을 검증한다. CRUD 는 DB 생성과 조회 함수를 모아두는 역할이다. 처음에는 이 구분이 익숙하지 않았지만, 이후 필드 불일치 문제를 겪으면서 세 계층을 따로 관리해야 하는 이유를 더 명확히 이해하게 됐다.

2.2 초기 테이블 관계

모델	역할	연결 관계
User	서비스 사용자 정보 저장	Image, Advertisement, History 를 여러 개 가짐
Image	업로드 이미지와 생성 결과 이미지 저장	사용자에 속하고, 광고의 입력·출력 이미지로 사용됨
Advertisement	광고 생성 요청과 결과 정보 저장	사용자, 입력 이미지, 출력 이미지와 연결됨
History	생성 성공·실패와 주요 작업 기록 저장	사용자와 광고 결과를 연결함

이 구조는 처음부터 완성된 구조는 아니었다. 진행하면서 username 컬럼, output_image_id 저장, 관리자 테이블, 구독과 크레딧 테이블이 계속 추가되었다. 그래도 초기에 User-Image-Advertisement-History 의 기본 흐름을 잡아둔 덕분에 이후 생성 결과 저장과 내 광고 조회 기능을 같은 기준으로 확장할 수 있었다.

2.3 구조를 잡으면서 신경 쓴 부분

첫째, 민감한 값이 API 응답으로 나가지 않게 했다. 예를 들어 비밀번호는 DB 에는 해시값으로 저장해야 하지만, 사용자 응답에는 포함되면 안 된다. 그래서 DB 모델과 응답 스키마를 분리했다.

둘째, 파일과 DB 를 함께 고려했다. 이미지는 실제 파일로 저장되지만, 서비스에서 조회하려면 DB 에 파일 경로, URL, 원본 파일명, 이미지 타입 같은 정보가 남아야 한다. 파일만 저장하면 히스토리와 연결할 수 없고, DB 만 저장하면 실제 이미지를 보여줄 수 없다.

셋째, 처음부터 너무 복잡한 구조를 만들지는 않았다. 프로젝트 기간이 제한되어 있었기 때문에 일단 MVP 에서 필요한 최소 관계를 잡고, 진행하면서 필요한 테이블과 필드를 추가하는 방식으로 접근했다.

3. 주요 구현 내용

3.1 이미지 업로드 API

7 월 3 일에는 이미지 업로드 API 를 구현했다. 사용자가 상품 사진을 올리면 서버에 파일을 저장하고, images 테이블에 업로드 정보를 기록하는 기능이다. 파일을 보내야 했기 때문에 일반 JSON 요청이 아니라 multipart/form-data 방식을 사용했다.

초기에는 jpg, jpeg, png, webp 확장자와 content-type 을 검사하고, 파일 크기 제한과 빈 파일 예외를 처리했다. 저장 파일명은 원본 파일명을 그대로 쓰지 않고 UUID 기반으로 생성했다. 원본 파일명을 그대로 사용하면 같은 이름의 파일이 충돌하거나 특수문자로 인해 경로 문제가 생길 수 있기 때문이다.

업로드 후에는 image_id, filename, content_type, image_url 을 응답하도록 했다. 프론트에서 image_url 로 이미지를 볼 수 있도록 /uploads 정적 파일 경로도 연결했다. 이 작업을 하면서 정상 업로드뿐 아니라 허용되지 않은 확장자, 파일 크기 초과, 빈 파일, 존재하지 않는 사용자 요청도 Swagger 에서 확인했다.

3.2 대시보드 요약 API

같은 날 프론트 대시보드에서 사용할 요약 API 도 추가했다. 대시보드에는 이번 달 생성 광고 수, 최근 작업일, 최근 광고 목록이 필요했다. 이를 위해 dashboard 용 CRUD 와 응답 스키마를 만들고 GET /dashboard/summary 형태로 제공했다.

처음에는 user_id 를 query parameter 로 받았지만, 이후 JWT 기반 사용자 흐름으로 바꾸면서 현재 로그인한 사용자 기준으로 조회하도록 수정했다. 이 변화는 단순히 요청 형식만 바꾼 것이 아니라, 사용자가 다른 사람의 대시보드 정보를 조회하지 못하도록 하는 방향으로 개선한 작업이었다.

3.3 광고 결과와 히스토리 저장

광고 생성 결과는 단순히 이미지 파일 하나가 아니라, 사용자가 다시 확인할 수 있는 기록으로 남아야 했다. 그래서 생성 결과 이미지는 Image 테이블에 image_type="generated"로 저장하고, Advertisement 에는 입력 이미지와 출력 이미지, 상품명, 스타일, 문구, 상태를 저장하도록 연결했다. History 에는 생성 성공 또는 실패 기록을 남겼다.

이 작업에서 중요했던 부분은 output_image_id 연결이었다. 모델에는 output_image_id 가 있었지만 실제 저장 로직에서 사용되지 않던 부분을 확인했고, 생성된 최종 이미지를 Image 로 저장한 뒤 Advertisement.output_image_id 와 연결했다. 덕분에 내 광고 목록이나 상세 화면에서 입력 이미지, 생성 이미지, 문구 정보를 함께 보여줄 수 있었다.

또 생성이 실패한 경우에도 History 에 실패 기록을 남기도록 했다. 실패 기록이 없으면 사용자는 왜 결과가 없는지 알 수 없고, 개발자도 어떤 요청에서 문제가 발생했는지 추적하기 어렵다.

3.4 히스토리 조회, 삭제, 다운로드

히스토리 기능은 사용자가 만든 광고 결과를 다시 확인하는 흐름이다. 단순히 histories 테이블만 조회하면 화면에서 필요한 생성 이미지와 문구 정보가 부족했다. 그래서 History 조회 응답에 Advertisement 와 입력·출력 이미지 정보를 함께 연결하는 방향으로 개선했다.

삭제 기능도 추가했다. 광고 결과는 생성 직후 이미 내 광고에 저장되므로 별도의 저장 버튼은 혼동을 줄 수 있다고 판단했고, 결과를 관리하는 동작은 내 광고 목록과 삭제 기능으로 정리했다. 삭제할 때는 본인 히스토리인지 확인한 뒤 연결된 생성 결과만 삭제하고, 입력 이미지는 무조건 지우지 않도록 했다.

프리미엄 사용자의 원본 다운로드 기능에서는 단순히 파일 경로가 존재하는지만 보지 않고, 현재 사용자가 해당 히스토리의 소유자인지와 활성 프리미엄 구독이 있는지를 함께 확인했다.

3.5 계정, 구독, 크레딧

중반 이후에는 서비스 운영에 필요한 계정, 구독, 크레딧 기능도 작업했다. 프로젝트에서는 실제 결제사가 연결된 상태가 아니었기 때문에 데모 결제 흐름으로 구독과 구매 내역을 저장했다. 이때 실제 카드 전체 번호나 CVC 를 저장하는 방식은 사용하지 않았다.

크레딧은 무료 사용자의 기본 크레딧, 프리미엄 월간 크레딧, 사용자가 구매한 크레딧, 관리자가 지급하는 보너스 크레딧으로 구분했다. 구독 갱신이나 환불이 있을 때 모든 크레딧이 섞이면 데이터가 꼬일 수 있기 때문에 별도 잔액으로 관리하는 방향이 필요했다.

광고 생성에 실패했을 때 크레딧이 그대로 차감되면 사용자가 불합리하게 느낄 수 있다. 그래서 생성 전에 차감하되, 생성 실패 시 사용했던 종류의 크레딧을 복구하는 로직을 같이 확인했다.

3.6 관리자 API 와 권한 관리

7 월 15 일부터는 관리자 페이지에서 사용할 백엔드 API 를 정리했다. 관리자는 회원, 구매 내역, 구독, 문의, 환불, 감사 로그를 확인해야 했다. 그래서 일반 사용자 API 와 관리자 API 를 /api/admin/* 형태로 분리하고, 관리자 역할에 따라 가능한 작업을 나눴다.

관리자 역할은 operator와 super_admin으로 구분했다. operator는 회원·구독·문의 조회와 문의 답변 같은 일반 운영 작업을 맡고, super_admin은 회원 정지, 플랜 변경, 환불, 관리자 계정 관리처럼 영향이 큰 작업을 할 수 있도록 했다. 또한 본인 계정을 직접 비활성화하거나 마지막 super_admin을 없애는 상황은 막았다.

관리자가 수행하는 주요 작업은 감사 로그에 남기도록 했다. 예를 들어 환불, 회원 상태 변경, 관리자 지정 같은 작업은 누가 어떤 작업을 했는지 남겨야 나중에 확인할 수 있다. 또 감사 로그 저장에 실패하면 앞선 변경도 함께 롤백되도록 트랜잭션을 고려했다.

4. 진행 중 겪은 문제와 해결 과정

4.1 스키마와 DB 모델 불일치

7월 2일 Swagger로 회원가입을 테스트하다가 UserCreate 스키마에는 username이 있는데 실제 User 모델에는 username 컬럼이 없는 문제를 발견했다. 처음에는 Swagger 요청 문제라고 생각했지만, 코드를 확인해 보니 API 요청 스키마와 DB 모델이 서로 맞지 않는 상태였다.

User 모델에 username 컬럼을 추가하고 auth 로직에서 username이 저장되는 흐름을 확인했다. 그런데 수정 후에도 Internal Server Error가 발생했다. 원인은 기존 SQLite 파일의 users 테이블에는 새 컬럼이 없었기 때문이다. 모델 코드를 바꿔도 기존 DB 파일의 테이블 구조가 자동으로 바뀌지는 않았다.

이 문제는 로컬 테스트 DB를 초기화한 뒤 다시 회원가입과 로그인을 확인하면서 해결했다. 이 경험 때문에 이후 create_all() 방식의 한계를 문서화하고, 후반에는 기존 DB에도 변경 사항을 순서대로 적용할 수 있는 마이그레이션 구조를 추가하게 됐다.

4.2 user_id 직접 입력에서 로그인 사용자 기준으로 변경

초기 이미지 업로드와 대시보드 API는 user_id를 요청에서 직접 받았다. 구현은 빠르지만 실제 서비스에서는 위험한 방식이다. 사용자가 다른 user_id를 넣으면 다른 사람의 데이터에 접근할 가능성이 생긴다.

그래서 JWT에서 현재 로그인한 사용자를 가져오는 구조를 사용하도록 주요 API를 수정했다. 이미지 업로드는 현재 로그인한 사용자 기준으로 저장하고, 대시보드는 현재 사용자 데이터만 조회하도록 했다. 광고 생성에서는 image_id가 현재 사용자의 이미지인지 확인한 뒤 진행하도록 했다. 다른 사용자의 이미지라면 403을 반환했다.

4.3 파일 검증 방식의 한계

처음 이미지 업로드는 확장자와 content-type 중심으로 검증했다. 하지만 실제 사용 과정에서는 확장자와 실제 이미지 형식이 다를 수 있었다. 특히 카카오톡에서 저장한 이미지나 아이폰 사진처럼 MIME 타입과 실제 포맷이 다르게 들어오는 경우가 있었다.

그래서 후반에는 Pillow 로 실제 파일 내용을 읽어 이미지인지 확인하고, 정상 이미지라면 서버에서 JPEG 또는 PNG 형식으로 정규화해서 저장하도록 개선했다. 템플릿 제품 이미지, 일반 광고 생성 이미지, 프로필 사진에 같은 정책을 적용했다. HEIC-HEIF 처리를 위해 pillow-heif 의존성도 추가했다. 다만 협업일지 기준으로 실제 VM 에서 아이폰 이미지 업로드까지 완전히 확인한 것은 아니어서 남은 검증 항목으로 두었다.

4.4 관리자 로그인과 일반 사용자 로그인 분리

관리자 페이지를 점검하면서 일반 사용자 로그인과 관리자 로그인이 섞일 가능성을 확인했다. 관리자 페이지는 일반 사용자 화면과 성격이 다르고, 관리자 계정은 운영 권한을 갖기 때문에 로그인 흐름을 분리해야 했다.

그래서 일반 사용자 로그인 은 /api/auth/login, 관리자 로그인 은 /api/auth/admin-login 으로 나누었다. 일반 사용자 계정으로 관리자 페이지에 들어가면 차단하고, 관리자 로그인에서는 활성화된 operator 와 super_admin 만 통과하도록 했다. 또한 관리자 로그인 실패 메시지가 화면에 보이지 않던 문제도 hidden 속성을 함께 해제하는 방식으로 수정했다.

4.5 관리자 DB 분리와 TOTP

7 월 22 일에는 관리자 DB 를 일반 사용자 DB 와 분리했다. DB 를 나눈다고 보안이 자동으로 해결되는 것은 아니지만, 관리자 계정, 로그인 실패 기록, 감사 로그, TOTP 설정 같은 관리 전용 데이터를 별도로 관리할 필요가 있다고 판단했다.

관리자 TOTP 도 추가했다. 관리자 계정에는 TOTP 활성화 여부와 암호화된 비밀키를 저장하고, 로그인 시 아이디와 비밀번호 외에 인증 앱의 6 자리 번호를 추가로 확인하도록 했다. TOTP 를 해제할 때도 현재 비밀번호와 인증 번호를 함께 확인하게 했다.

4.6 DB 무결성과 파일 경로 검증

프로젝트 후반에는 전체 백엔드를 다시 보면서 DB 무결성과 파일 접근 문제를 점검했다. SQLite 는 설정하지 않으면 외래키 제약이 실제로 적용되지 않을 수 있어 DB 연결 시 외래키 검사를 활성화했다. 또 회원탈퇴 후에도 Refresh Token, 보너스 크레딧, 이메일 인증 기록이 남을 수 있어 삭제 범위를 보완했다.

파일 경로도 중요한 문제였다. DB 에 저장된 파일 경로를 그대로 믿으면 허용된 결과 이미지 폴더 밖의 파일에 접근할 위험이 있다. 그래서 관리자 광고 파일 조회나 사용자 다운로드에서 실제 경로가 허용된 결과 폴더 내부인지 확인한 뒤에만 접근하도록 검증을 추가했다.

근거: 07/27 협업일지: DB 무결성, 마이그레이션, 외래키, 파일 경로 검증, 임시 업로드 이미지 정리

5. 전환 및 배포 정리

목표는 SMTP 승인이 끝나기 전에도 인증 관련 기능을 테스트할 수 있게 하고, 기존 systemd 방식으로 실행하던 Next.js 와 FastAPI 일반 백엔드를 Docker Compose 기반으로 전환하는 것이었다. 다만 GPU 를 사용하는 이미지 생성 워커까지 Docker 로 옮기지는 않았다. 모델 파일과 CUDA 환경이 이미 구성되어 있었기 때문에, 일반 웹 서비스와 생성 서버를 분리해서 운영하는 혼합 구조로 정리했다.

5.1 SMTP 승인 전 임시 인증 모드

SMTP 발송 승인이 완료되지 않으면 이메일 인증번호 전송과 비밀번호 재설정 메일을 실제로 테스트하기 어렵다. 그렇다고 기존 이메일 인증 코드를 삭제하거나 임시 코드를 따로 만들면 나중에 다시 복구할 때 문제가 생길 수 있었다.

그래서 환경변수로 실제 인증 방식과 임시 테스트 방식을 전환할 수 있게 했다.

EMAIL_VERIFICATION_REQUIRED=false 일 때는 회원가입 시 이메일 인증번호 검사를 임시로 생략한다. 프론트도 서버 설정을 확인해 이메일 인증이 비활성화된 경우 인증번호 발송 버튼과 입력 화면을 숨기도록 했다. 비밀번호 찾기는 DEMO_PASSWORD_RESET_ENABLED=true 설정으로 실제 메일 없이 재설정 흐름을 테스트할 수 있게 했다.

중요한 점은 기존 인증 코드를 지우지 않았다는 것이다. SMTP 승인이 끝나면 코드를 다시 고치지 않고 환경변수만 운영값으로 변경해 원래 이메일 인증 방식으로 돌아갈 수 있게 했다.

5.2 Docker 전환 방향

기존에는 VM 에서 systemd 서비스가 Next.js 와 FastAPI 를 각각 직접 실행하고 있었다. systemd 방식은 서버에 설치된 Python 가상환경이나 Node.js 환경을 그대로 사용하므로, 서버 패키지 상태가 바뀌면 실행 환경도 같이 흔들릴 수 있다. Docker 는 실행에 필요한 코드와 패키지를 이미지로 묶어 실행하므로 환경을 더 일정하게 유지하기 쉽다.

최종적으로 다음과 같은 혼합 구조로 정리했다. Docker 에서는 Next.js 프론트엔드와 FastAPI 일반 백엔드를 실행하고, GPU 이미지 생성 워커는 기존 systemd 방식으로 유지한다. 외부 사용자는 기존처럼 Nginx 와 도메인을 통해 접속하고, Nginx 가 내부의 3000 번 프론트와 8000 번 백엔드로 요청을 넘긴다.

실행 방식	서비스	이유
Docker	Next.js 프론트엔드	프론트 실행 환경을 이미지로 고정하고 재배포하기 쉽게 하기 위해
Docker	FastAPI 일반 백엔드	로그인, 계정, 결제, 히스토리 등 일반 API 를 가벼운 이미지로 실행하기 위해

systemd	GPU 이미지 생성 워커	모델 파일과 CUDA 환경을 유지하고, 단기간에 무리하게 Docker 로 옮기지 않기 위해
Nginx	외부 요청 프록시	도메인과 HTTPS 구조를 유지하고 컨테이너 포트를 직접 노출하지 않기 위해

5.3 Docker 설치와 웹 스택 구성

VM 은 Debian GNU/Linux 13 이었고 Docker 가 설치되지 않은 상태였다. Docker 와 Docker Compose 를 설치한 뒤 docker compose version, docker run hello-world, systemctl is-active docker 로 설치 상태를 확인했다. Docker Compose 설정은 docker compose config -q 로 검사했고 결과 코드 0 으로 정상 통과했다.

FastAPI 일반 백엔드는 backend-web 컨테이너로 8000 번에서 실행하고, Next.js 프론트는 frontend 컨테이너로 3000 번에서 실행하도록 구성했다. 두 포트는 외부 전체에 열지 않고 127.0.0.1 에만 연결했다. 외부 요청은 반드시 Nginx 를 거치도록 해서 컨테이너 포트가 인터넷에 직접 노출되지 않게 했다.

DB 와 이미지 파일은 컨테이너 내부가 아니라 VM 의 runtime 경로에 저장하도록 했다. 컨테이너는 삭제하고 다시 만들 수 있기 때문에, app.db, admin.db, 업로드 이미지, 생성 결과 이미지를 컨테이너 안에 두면 배포 과정에서 데이터가 사라질 수 있다. 그래서 runtime/data 와 runtime/results 를 영구 저장 경로로 두었다.

5.4 웹 백엔드 의존성 분리

Docker 이미지를 처음 빌드하는 과정에서 VM 디스크 공간 부족 문제가 발생했다. 확인해 보니 /opt/adnova/models 가 약 39GB, 기존 백엔드 가상환경이 약 5.9GB 를 사용하고 있었다. 모델 캐시는 GPU 생성 워커가 실제로 사용하는 파일이라 삭제하지 않기로 했다.

문제는 일반 FastAPI 웹 백엔드 Docker 이미지에도 Torch, Diffusers, Transformers, CUDA 관련 패키지가 포함되고 있었다는 점이다. 로그인, 회원, 결제, 히스토리 같은 일반 API 는 무거운 생성 모델 패키지를 직접 사용할 필요가 없다. 그래서 requirements.web.txt 를 분리해 웹 백엔드는 일반 API 에 필요한 가벼운 CPU 패키지만 설치하도록 수정했다. GPU 워커는 기존 8100 번 systemd 서비스에서 모델 패키지를 유지하도록 했다.

5.5 기존 DB 와 이미지 데이터 복원

Docker 실행 후 기존 계정으로 로그인되지 않는 문제가 있었다. 처음에는 DB 가 초기화된 것처럼 보였지만, 실제 원인은 systemd 백엔드와 Docker 백엔드가 서로 다른 SQLite 파일을 보고 있었기 때문이다. 기존 systemd 는 backend/data 를 사용하고, Docker 는 runtime/data 를 사용했다. Docker 가 새 위치에 빈 DB 를 만들면서 기존 데이터가 사라진 것처럼 보인 것이다.

배포 전 백업 폴더에서 운영 중 사용하던 app.db 와 admin.db 를 확인했고, Docker 가 새로 만든 DB 도 따로 백업한 뒤 기존 운영 DB 를 runtime/data 로 복원했다. 그 결과 Docker 환경에서도 기존 사용자와 관리자 데이터를 다시 사용할 수 있게 했다.

기존 생성 이미지도 비슷한 문제가 있었다. 기존 결과 파일은 backend/results 에 있었고 Docker 는 runtime/results 를 사용했다. DB 에는 systemd 환경의 절대경로가 저장되어 있어 컨테이너 내부에서 접근할 수 없었다. 이 문제는 migrate_docker_results.py 마이그레이션 도구를 추가해 해결했다. 도구는 실행 전 DB 를 백업하고, 기존 결과 파일을 runtime/results 로 복사한 뒤 DB 이미지 경로를 /app/results/... 형식으로 변경한다. 마이그레이션 결과 전체 대상 379 개 중 251 개는 이전했고, 128 개는 기존 위치에도 원본 파일이 없어 복구하지 못했다. 이 부분은 억지로 성공 처리하지 않고 남은 이슈로 분리했다.

5.6 Docker 백엔드와 GPU 워커 연결

Docker 백엔드가 이미지 생성을 위해 host.docker.internal:8100 으로 요청했지만 연결이 거부되는 문제가 있었다. 원인은 GPU 워커가 127.0.0.1:8100 에만 바인딩되어 있었기 때문이다. 호스트에서 실행되는 서비스가 127.0.0.1 에만 묶여 있으면 Docker 브리지 네트워크에서는 접근할 수 없다.

생성 워커를 0.0.0.0:8100 으로 바인딩하고, Docker Compose 에는 host.docker.internal:host-gateway 설정을 적용했다. 다만 8100 번 포트는 Nginx 나 GCP 외부 방화벽에 열지 않았다. Docker 컨테이너에서는 접근할 수 있지만 외부 사용자가 직접 호출할 수 없도록 한 것이다.

5.7 systemd 와 Docker 포트 충돌

Docker 컨테이너가 실행되지 않는 또 다른 원인은 기존 systemd 서비스가 같은 포트를 사용하고 있었기 때문이다. 기존 adnova.service 는 8000 번을, adnova-frontend.service 는 3000 번을 사용했다. Docker 도 같은 포트를 사용해야 했기 때문에 기존 일반 백엔드와 프론트 systemd 서비스를 중지하고 비활성화했다.

그런데 잠시 후 8000 번 충돌이 다시 발생했다. 확인 결과 기존 adnova-autodeploy.timer 가 주기적으로 develop 을 받아 systemd 백엔드를 다시 시작하고 있었다. Docker 방식과 기존 자동 배포 방식이 충돌한 것이다. 그래서 해당 타이머도 중지하고 비활성화했다. 포트 바인딩 실패 이후 Docker 컨테이너의 네트워크 연결 정보가 비어 있는 문제는 docker compose up -d --force-recreate 로 컨테이너를 강제로 다시 생성해 해결했다.

5.8 Docker 자동 배포 구조 준비

기존 자동 배포는 develop 브랜치를 가져온 뒤 systemd 프론트와 백엔드를 재시작하는 방식이었다. Docker 전환 후 이 방식이 남아 있으면 다시 포트 충돌을 만들 수 있다. 그래서 Docker 전용 자동 배포 구조를 준비했다.

새 자동 배포는 main 브랜치만 추적하도록 했다. 작업 트리에 수정사항이 남아 있거나 fast-forward 방식으로 최신 코드를 받을 수 없으면 배포를 중단한다. 배포 과정은 main 최신 상태 확인, Docker 이미지 빌드, backend-web 과

frontend 실행, 8000 health 확인, 3000 프론트 확인, 8100 생성 워커 health 확인, 마지막 성공 커밋 기록 순서로 구성했다. 실패하면 다음 5 분 주기 실행에서 다시 시도할 수 있도록 systemd service 와 timer 도 준비했다.

다만 main 에 정식 병합하지 않았기 때문에 Docker 자동 배포 타이머는 최종 활성화하지 않았다. 실제 브라우저 광고 생성, 크레딧 차감, 내 광고 저장, 관리자 페이지 이미지 표시까지 E2E 검증이 남아 있었기 때문이다.

6. 검증 방식과 협업 방식

6.1 검증 방식

백엔드 작업은 눈에 바로 보이지 않는 경우가 많아서, 가능한 한 실행 결과를 남기려고 했다. 초기 DB 구조는 smoke_create_tables.py 로 테이블 생성 여부를 확인했다. API 작업은 Swagger 에서 정상 요청과 예외 요청을 함께 확인했다. Docker 작업은 명령어 결과, 포트 상태, health 응답으로 나눠 확인했다.

특히 Docker 전환에서는 단순히 컨테이너가 켜지는지만 보지 않았다. 기존 DB 가 제대로 복원되는지, 관리자 OTP 가 동작하는지, 결과 이미지 경로가 컨테이너 내부에서 접근 가능한지, Docker 백엔드가 8100 GPU 워커에 접근하는지, 기존 systemd 자동 배포가 다시 포트를 점유하지 않는지까지 확인했다.

작업	검증 방법
DB 초기 구조	smoke_create_tables.py 실행, users/images/advertisements/histories 테이블 생성 확인
회원가입 정합성	Swagger 에서 signup/login 테스트, username 응답 확인
이미지 업로드	정상 업로드, 잘못된 확장자, 빈 파일, 크기 초과 응답 확인
광고 이력	생성 결과가 Image, Advertisement, History 에 연결되는지 확인
관리자 기능	권한별 접근 가능 여부, 환불·회원 상태 변경·감사 로그 흐름 확인
Docker 전환	docker compose config, health check, 포트 충돌, DB 복원, 8100 워커 연결 확인

6.2 협업 방식

작업은 기능 브랜치에서 진행하고 develop 으로 PR 을 올리는 방식으로 했다. 초반에는 base 브랜치를 잘못 잡아 PR 을 다시 만든 경험도 있었다. 이후에는 develop 기준으로 작업을 진행하고, main 은 최종 운영 배포용으로 구분하는 방향을 더 명확히 이해했다.

코드 수정 범위도 한 번에 너무 크게 넓히지 않으려고 했다. 예를 들어 username 컬럼 문제는 User 모델 수정에 집중했고, asset_id 검증은 스키마와 서비스 검증 기준을 맞추는 작업으로 제한했다. 관리자 기능이나 Docker 작업처럼 범위가 큰 작업은 협업일지에 문제, 원인, 해결, 남은 작업을 나누어 기록했다.

7. 남은 한계와 개선 방향

7.1 SQLite 유지의 한계

현재 프로젝트는 최종 데모와 단일 VM 운영을 고려해 SQLite 를 유지했다. 하지만 SQLite 는 동시 쓰기가 많아지는 운영 환경에는 한계가 있다. 사용자 수가 늘고 결제, 크레딧, 관리자 작업이 동시에 발생한다면 PostgreSQL 로 전환하는 것이 더 적합하다.

7.2 마이그레이션 체계

초기에는 create_all() 중심으로 테이블을 만들었다. 이 방식은 처음 테이블을 만들 때는 편하지만, 기존 DB 에 컬럼을 추가하거나 구조를 바꿀 때 자동으로 반영되지 않는다. 후반에 버전 기반 마이그레이션 구조를 추가했지만, 장기적으로는 Alembic 같은 정식 마이그레이션 도구를 도입하는 것이 좋다.

7.3 실제 결제와 이메일

결제는 데모 흐름으로 구현했다. 실제 서비스라면 PG 승인, 결제 실패, 부분 환불, 웹훅 서명 검증, 영수증 처리까지 필요하다. 이메일도 7 월 28 일 기준으로 SMTP 승인 전 임시 모드를 사용했기 때문에, SMTP 승인이 완료되면 실제 이메일 인증과 비밀번호 재설정 메일 수신 테스트를 해야 한다.

7.4 파일 저장소와 백업

현재 DB 와 이미지는 VM 의 runtime 경로에 저장한다. 컨테이너 재생성에는 대응할 수 있지만, VM 장애에는 취약하다. 운영 서비스를 생각하면 Cloud Storage 나 S3 같은 객체 스토리지, 정기 백업, 파일 누락 검증, 복구 절차가 필요하다.

8. 마무리

이번 프로젝트에서 내가 맡은 역할은 백엔드 및 DevOps 였다. 처음에는 DB 모델, 스키마, CRUD 를 잡는 일부터 시작했고, 이후 이미지 업로드, 대시보드, 광고 결과 저장, 히스토리, 관리자 기능, 크레딧, Docker 배포 정리까지 작업 범위가 넓어졌다.

가장 많이 배운 점은 백엔드 작업이 단순히 API 를 하나 추가하는 일이 아니라는 것이다. 스키마와 DB 모델이 맞는지, 기존 SQLite 파일 구조가 따라오는지, 파일 경로가 안전한지, 생성 결과가 여러 테이블에 일관되게 저장되는지, 관리자 권한이 잘 나뉘는지, 배포 후 기존 데이터가 유지되는지까지 같이 봐야 했다.

아직 부족한 점도 많다. PostgreSQL 전환, Alembic 도입, 실제 결제와 이메일 연동, E2E 테스트, Docker 자동 배포 활성화, 파일 백업 정책은 더 보완해야 한다. 그래도 프로젝트 기간 동안 매일 문제를 기록하고, 원인을 확인하고, 해결한 내용과 남은 작업을 구분해 간 점은 개인적으로 의미 있는 경험이었다.

6. 사용자·관리자 프론트엔드와 QA — 정보

원문 개인보고서의 본문·표를 통합 문서 체계에 맞춰 수록하였다. 담당: 정보

프로젝트와 개인 담당 범위

AdNova 서비스에서 담당한 역할을 중심으로 정리하였다.

서비스 개요

AdNova는 제품 사진과 요청 문장을 입력하면 AI가 광고 이미지와 플랫폼별 문구를 생성하는 서비스다. 사용자가 광고를 만들고 저장·공유하는 과정과 운영자가 회원·결제·문의 등을 관리하는 과정으로 구성된다.

개인 담당 범위

구분	수행 내용
사용자 프론트엔드	로그인부터 이미지 업로드, 광고 생성, 저장·공유까지 전체 흐름 구현
관리자 페이지	회원·구독·결제·환불·문의·관리자 계정 관리 화면 구성
백엔드 연동	관리자 인증과 실제 데이터 조회·변경 API 연결
QA	기능·모바일·외부 로그인·결제·AI 결과 검증 및 기록

수행 방향

초기 화면 구현에서 시작해 실제 백엔드 연동과 관리자 운영 화면, QA까지 범위를 확장했다. 단순 화면 제작보다 사용자 흐름과 운영 흐름을 연결하는 데 초점을 맞췄다.

사용자 프론트엔드 구현

로그인부터 광고 저장·공유까지 이어지는 이용 흐름

주요 구현 내용

- 로그인·회원가입·비밀번호 찾기·SNS 로그인 화면과 입력 검증
- 이미지 업로드·미리보기·변경, 요청 문장과 6종 스타일 선택
- 광고 생성 단계 안내, 재생성, 중복 클릭 방지
- 내 광고·히스토리 조회, 삭제, SNS 공유

구현 결과

Streamlit 의 상태 중심 화면을 브라우저 화면 전환과 사용자 입력 상태로 재구성했다. 초기에는 백엔드 없이도 화면을 확인할 수 있게 구성했고, 이후 인증·이미지 업로드·광고 생성·히스토리 API 와 연결할 수 있도록 요청 구조를 정리했다.

관리자 페이지와 백엔드 연동

서비스 이용 영역과 운영 영역을 분리한 관리자 구조

관리자 페이지 구성

- 일반 사용자 페이지와 /admin/ 관리자 페이지 분리
- 회원 상태와 구독 플랜 조회·변경
- 주문별·회원별 결제 내역과 환불 신청 관리
- 1:1 문의 답변 및 관리자 계정·감사 로그 관리
- 임시 데이터 화면을 실제 관리자 로그인과 API 흐름으로 연결

협업 결과

관리자 페이지 프론트엔드와 백엔드 연동을 하나의 작업 흐름으로 정리해 PR #113 으로 공유했다. 이후 Next.js 이전 과정에서는 사용자 페이지의 색상과 컴포넌트 스타일에 맞춰 관리자 UI 를 통일해 PR #196 으로 반영했다.

QA 수행 결과

기능과 AI 결과를 사용자 관점에서 검증

구분	수행 내용
총 테스트	127 건
PASS	109 건
BLOCK	4 건
NOT TESTED	14 건
진행률	89%
AI 품질 평균	4.6 / 5

주요 확인 사항

- 소셜 로그인 취소 후 로딩 상태가 유지되는 문제
- 모바일 내비게이션 텍스트가 잘리거나 줄바꿈되는 문제

- 결제 입력값 검증과 SNS 공유 방식 개선 필요
- 일반 생성과 템플릿 생성 결과의 품질 차이

BLOCK 처리 기준

이메일 발송, 네이버 OAuth 테스트 등록, PG 계약처럼 외부 환경이 준비되지 않아 검증할 수 없는 항목은 오류로 단정하지 않고 BLOCK으로 구분했다. 가능한 기능은 대체 경로로 확인하고, 미검증 사유를 QA 시트에 기록했다.

수행 성과와 배운 점

주요 성과

- Streamlit 중심 기능 흐름을 HTML/JavaScript와 Next.js 사용자 화면으로 전환
- 관리자 운영 영역을 별도 페이지와 API 구조로 정리
- 관리자 인증과 회원·결제·환불·문의 데이터 흐름 연결
- 관리자 UI를 사용자 페이지 디자인과 통일
- QA 결과를 팀이 확인할 수 있는 재현 가능한 기록으로 정리

배운 점

프론트엔드는 화면을 꾸미는 역할에 그치지 않고, 백엔드의 데이터와 권한, 서비스 정책과 오류 상태를 사용자가 이해할 수 있는 경험으로 바꾸는 역할임을 배웠다. 또한 기능 구현 후 실제 사용자 흐름으로 다시 검증해야 누락된 동선과 운영 문제를 발견할 수 있었다.

향후 과제

외부 인증·이메일·결제 환경이 준비된 뒤 BLOCK 항목을 재검증하고, 상세 광고 편집과 모바일 내비게이션, SNS 공유 방식을 개선할 필요가 있다.

관련 결과물

구분	수행 내용
PR #113	관리자 페이지 프론트엔드 및 백엔드 연동
PR #196	Next.js 관리자 페이지 디자인 통일
PR #222	무료 플랜 워터마크와 광고 이미지 표시 개선
QA Sheet	기능·AI 품질 테스트 결과 기록

7. 통합 보안·데이터 무결성·검증 결과

- 비밀번호 bcrypt 해시, JWT-Refresh Token, 관리자 TOTP
- MIME 표기만 신뢰하지 않는 실제 이미지 바이너리 검증과 JPEG/PNG 정규화
- 허용된 결과 폴더 내부인지 확인하는 경로 샌드박스
- 관리자·사용자 인증과 DB 분리, 최소 권한 적용
- 환불·상태 변경·감사 로그 실패 시 트랜잭션 롤백
- 회원 탈퇴 시 토큰·크레딧·이메일 인증 연계 데이터 정리

7.1 최종 QA

QA 결과	수량
PASS	109
BLOCK	4
추가 테스트 필요	14
AI 품질 평가	4.6 / 5

2026-07-27 최종 QA 기준 전체 127 건 중 113 건을 수행했고 PASS 109 건, FAIL 0 건, BLOCK 4 건, NOT TESTED 14 건, 진행률 89%를 기록했다. BLOCK은 이메일 발송 환경, Naver OAuth 테스터, 실제 PG 계약처럼 외부 조건이 준비되지 않아 검증을 보류한 항목이다.

8. 종합 한계와 다음 단계

영역	현재 한계	다음 단계
AI	생성 약 78 초, 손-held 강제 SKU, 로컬 가중치 비상업 라이선스	상업 허용 모델·큐·비동기 상태·대규모 홀드아웃
데이터	SQLite·단일 VM 파일 저장	PostgreSQL·Alembic·S3·백업·복구 훈련
외부 연동	데모 결제, OAuth-SMTP 승인 의존	PG 웹훅·앱 심사·실환경 E2E
품질	수동 QA 중심	회귀 자동화·모니터링·접근성·부하 테스트

AI 측면에서는 상업 허용 로컬 모델, 영속 GPU 큐, 비동기 상태 계약과 대규모 홀드아웃이 필요하다. 데이터 측면에서는 PostgreSQL-Alembic-S3-백업-복구 체계가 필요하며, 외부 연동은 OAuth-SMTP-PG 승인 후 실환경 E2E 로 재검증해야 한다. 품질 측면에서는 수동 QA 를 회귀 자동화-모니터링-접근성-부하 테스트로 확장해야 한다.

8.1 결론

AdNova 는 생성 모델 데모를 넘어 사용자가 실제로 가입하고 상품 사진을 올려 광고를 만들고 저장·공유하며, 운영자가 회원·결제·문의·환불을 관리할 수 있는 서비스 구조를 완성했다. 팀은 제한된 자원에서 로컬과 상용 API 를 목적별로 배치하고, 실패를 프롬프트·권한·데이터·배포 규칙으로 전환했다.

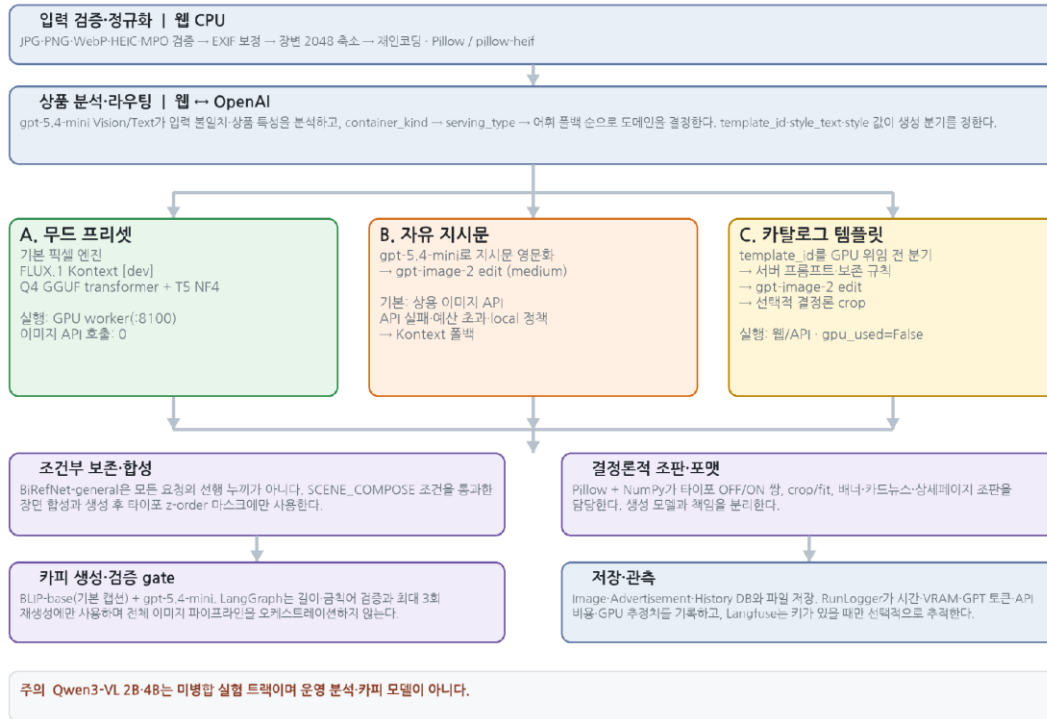
부록. 개인보고서 증빙 이미지

원문 개인보고서에 포함된 실험 결과 및 구현 증빙 이미지를 작성자별로 수록한다. 이미지의 세부 설명은 해당 본문과 원문 캡션을 함께 참조한다.

부록 한의정. 한의정 보고서 이미지

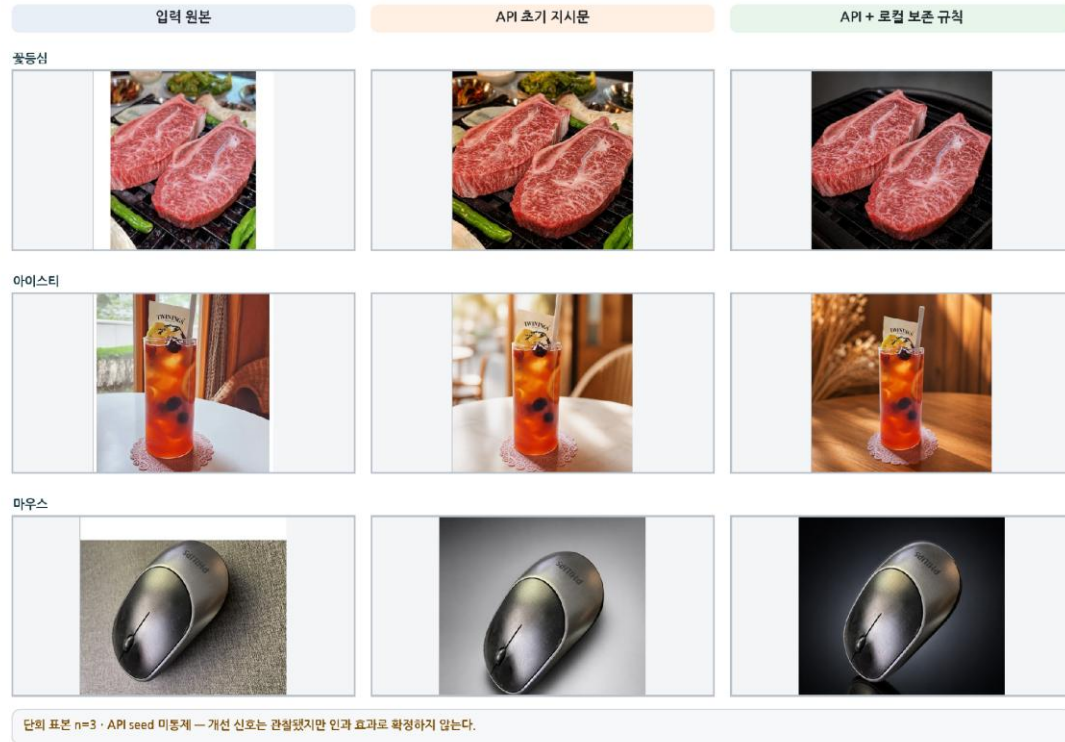
AdNova 운영 생성 파이프라인 — 3개 생성 분기와 공통 후단

운영 정본: servingtype · POST /api/ads/generate



한의정 원문 증빙 이미지 1

로컬에서 만든 보존 규칙을 같은 API 엔진에 이식한 탐색 비교



한의정 원문 증빙 이미지 2

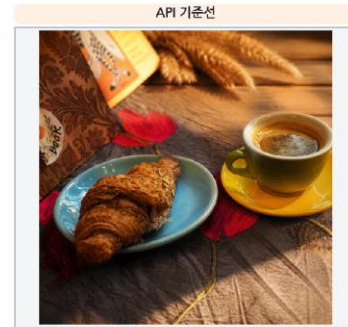
사례 A — 딸기 생크림 케이크: 제품 중심의 클린 재연출



같은 원본에서 생성 · local은 인물 휴대문·카페 접수를 제거하고 케이크 층과 딸기를 유지했다.

한의정 원문 증빙 이미지 3

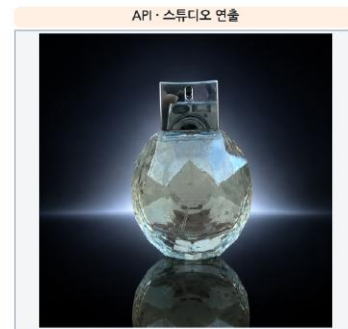
사례 B — 크루아상: 원형 보존과 배경 단순화



local은 크루아상·청록 접시·초록 컵의 정제성을 유지하면서 불필요한 소품을 뺐다.

한의정 원문 증빙 이미지 4

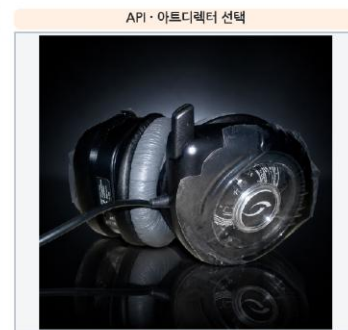
사례 C — 향수: 보존형 clean cutout과 API 연출의 차이



local은 병 형상을 보존한 clean cutout, API는 반사조영까지 만든다. 평가 기준에 따라 선호가 갈릴 수 있다.

한의정 원문 증빙 이미지 5

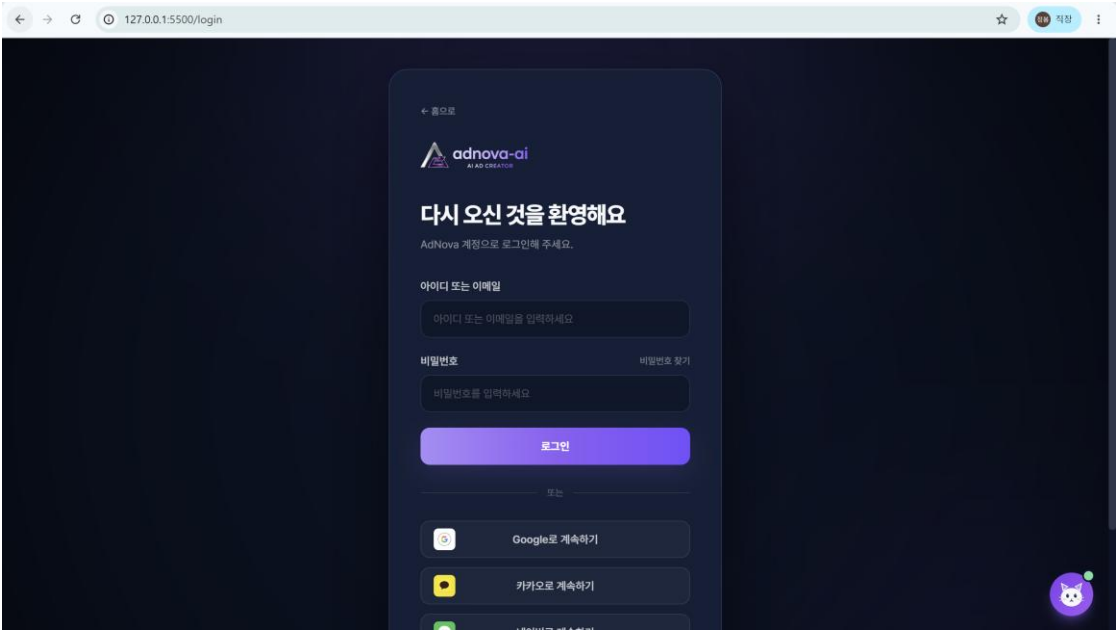
반례 — 헤드폰: API가 보존과 광고 연출을 함께 달성



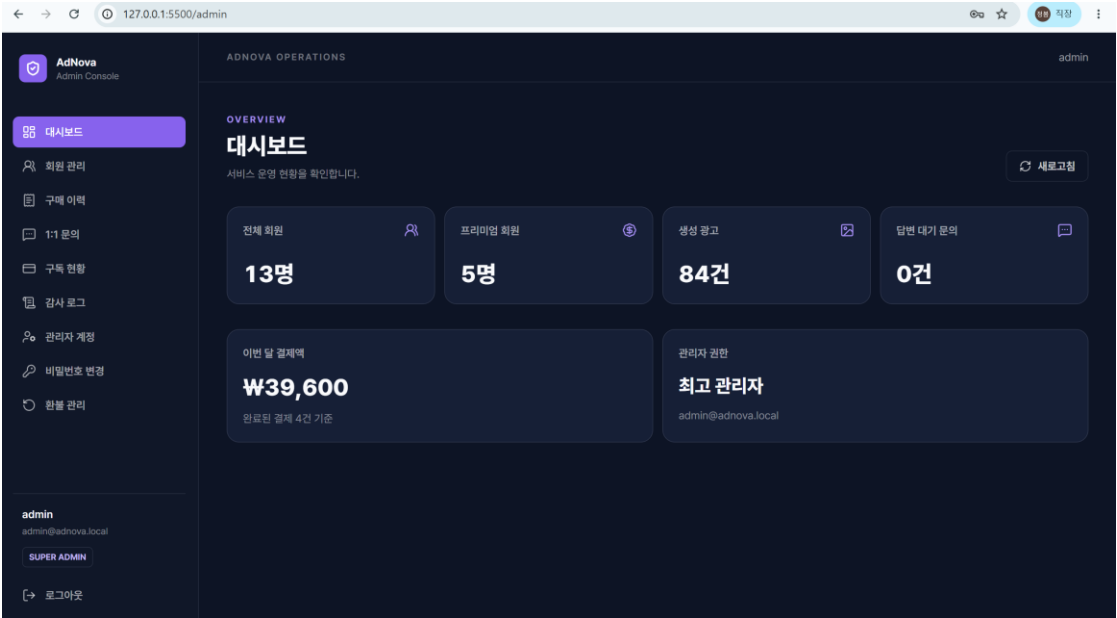
사울이라고 local이 항상 유리하지 않다. 헤드폰은 API가 원형을 유지하면서 조명·반사구도를 완성했다.

한의정 원문 증빙 이미지 6

부록 정보. 정보 보고서 이미지



정보 원문 증빙 이미지 1



정보 원문 증빙 이미지 2

	A	B	C	D	E	F	G	H	I	J	K
1	Bug ID	Severity	Priority	Category	Description	Reproduce Steps	Status	Assignee	Fixed Version	Date Four	
	BUG-003	High	High	AUTH	회원가입 시 이메일 인증번호 발송 실패로 회원가입 불가	3. 이메일 인증번호 발송 버튼 클릭 4. 인증번호 발송 실패 확인	Open			2026-07-22	

정보 원문 증빙 이미지 3